

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Interconnects (Part 3)

Instructor: Prof. Joshua San Miguel

Some slides adopted from Natalie Enright Jerger

Last Lecture

- Bisection bandwidth
- Path diversity
- Direct topologies
- Torus

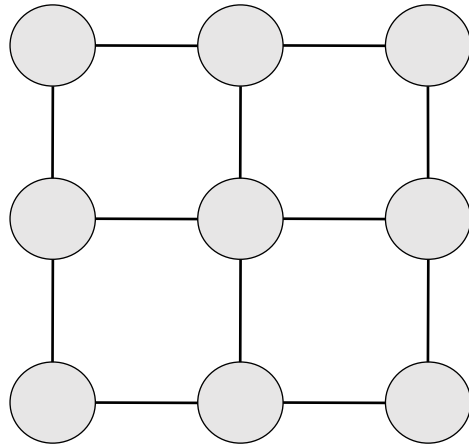
This Lecture

- Mesh
- Indirect topologies
- Butterfly
- Deterministic routing
- Oblivious routing

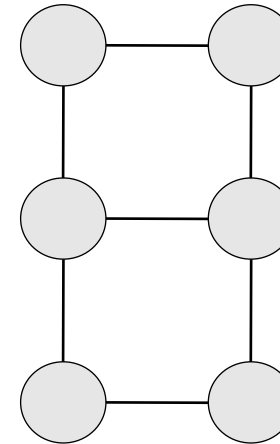
Direct Topologies – Mesh

k-ary n-mesh:

- Torus with no wrap-around links
- No edge symmetry; prone to load imbalance (i.e., higher demand for central links)
- N routers = k^n (or $k_1 \times \dots \times k_n$ if not equilateral)



3-ary 2-mesh



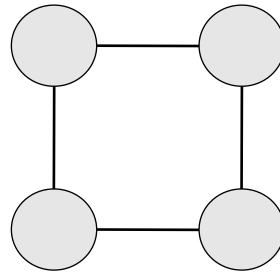
2,3-ary 2-mesh



Direct Topologies – Mesh

k-ary n-mesh:

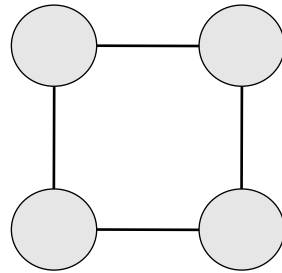
- Torus with no wrap-around links
- No edge symmetry; prone to load imbalance (i.e., higher demand for central links)
- $N \text{ routers} = k^n$ (or $k_1 \times \dots \times k_n$ if not equilateral)



Direct Topologies – Mesh

k-ary n-mesh:

- Torus with no wrap-around links
- No edge symmetry; prone to load imbalance (i.e., higher demand for central links)
- N routers = k^n (or $k_1 \times \dots \times k_n$ if not equilateral)

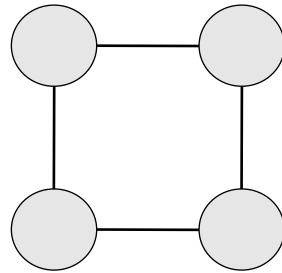


2-ary 2-mesh

Direct Topologies – Mesh

k-ary n-mesh:

- Torus with no wrap-around links
- No edge symmetry; prone to load imbalance (i.e., higher demand for central links)
- N routers = k^n (or $k_1 \times \dots \times k_n$ if not equilateral)



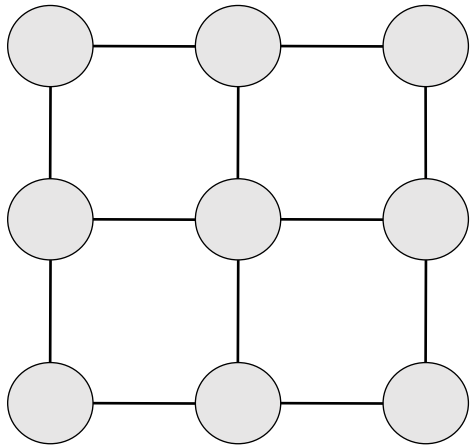
2-ary 2-mesh

4-ary 1-cube (i.e., ring)

Direct Topologies – Mesh

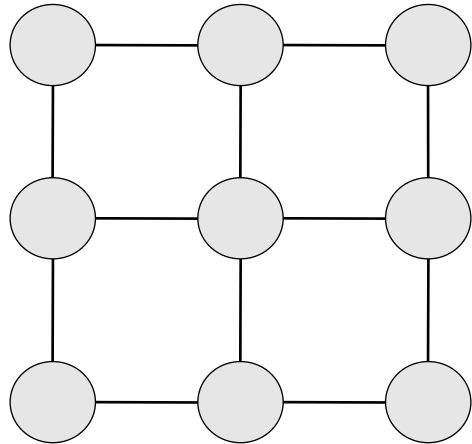
Degree:

- Minimum degree = n
- Maximum degree = $2n$



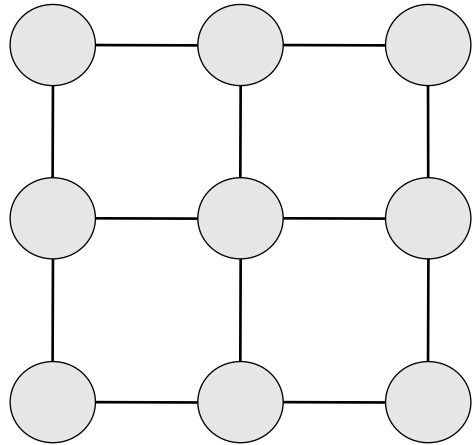
Direct Topologies – Mesh

Network Diameter:



Direct Topologies – Mesh

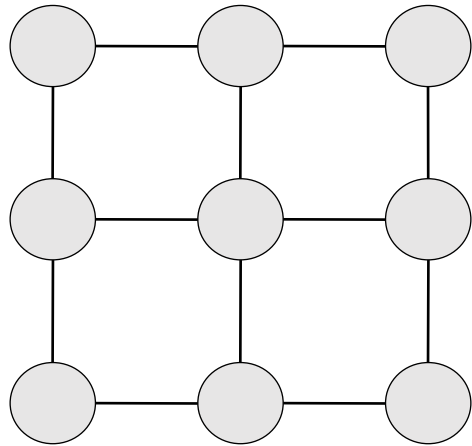
Network Diameter:



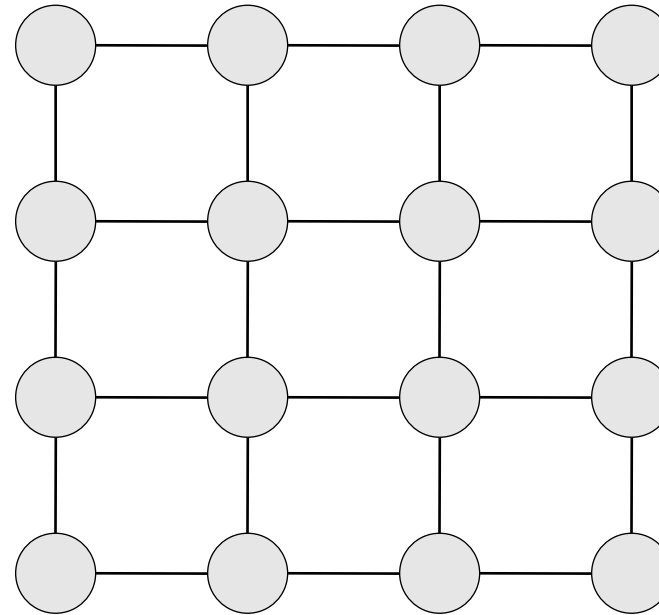
4 hops

Direct Topologies – Mesh

Network Diameter:

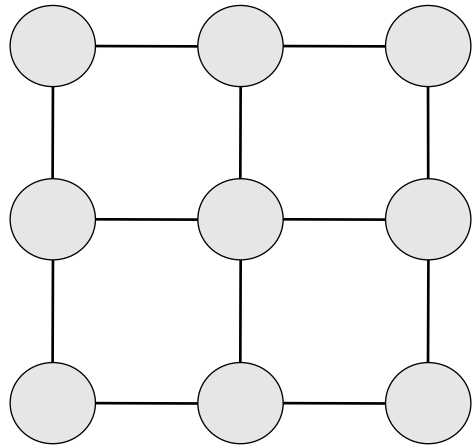


4 hops

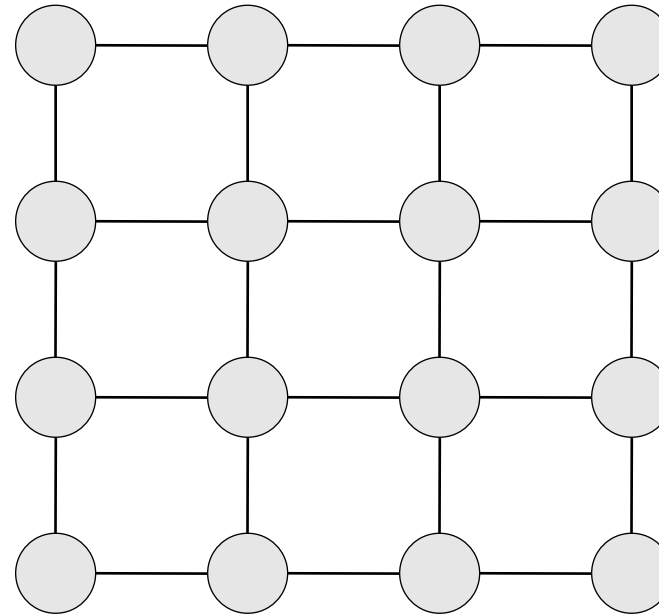


Direct Topologies – Mesh

Network Diameter:



4 hops



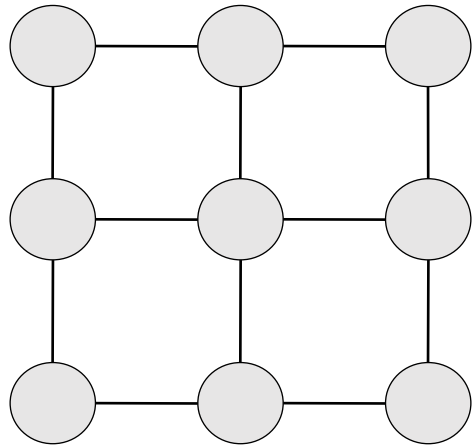
6 hops



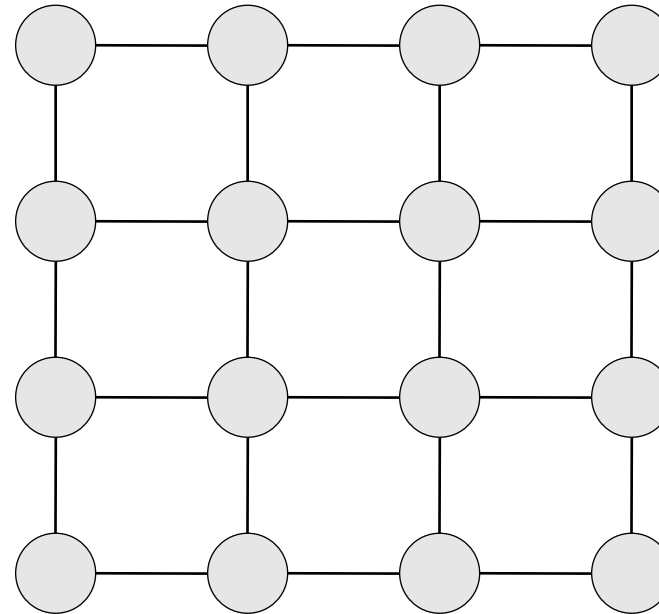
Direct Topologies – Mesh

Network Diameter:

➤ Diameter = $(k_1 - 1) + \dots + (k_n - 1)$



4 hops



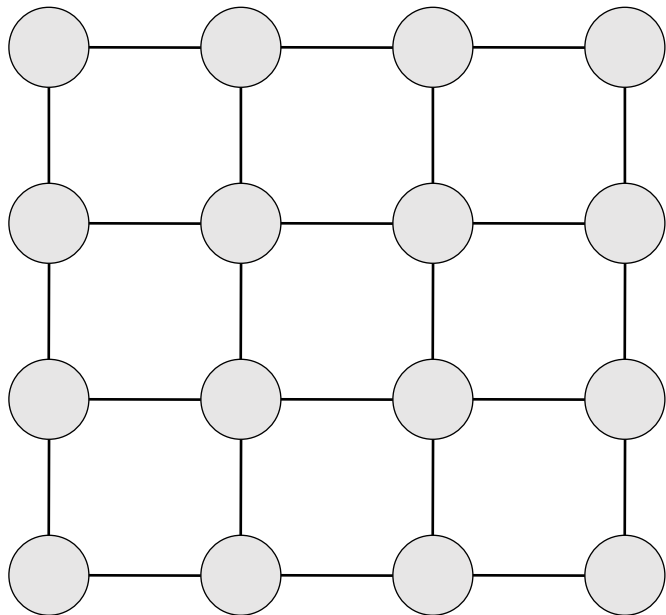
6 hops



Direct Topologies – Mesh

Bisection Bandwidth:

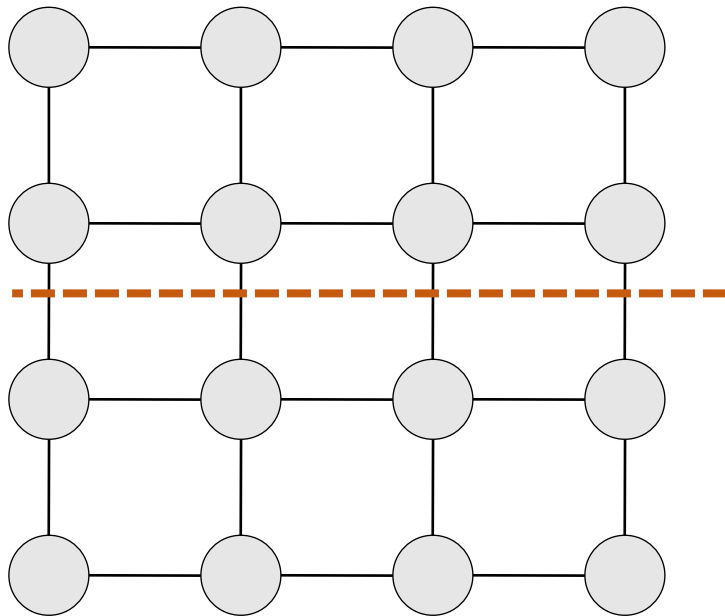
- Bisection bandwidth = $2N/k$ unidirectional links, largest k
- Half of torus bisection bandwidth



Direct Topologies – Mesh

Bisection Bandwidth:

- Bisection bandwidth = $2N/k$ unidirectional links, largest k
- Half of torus bisection bandwidth



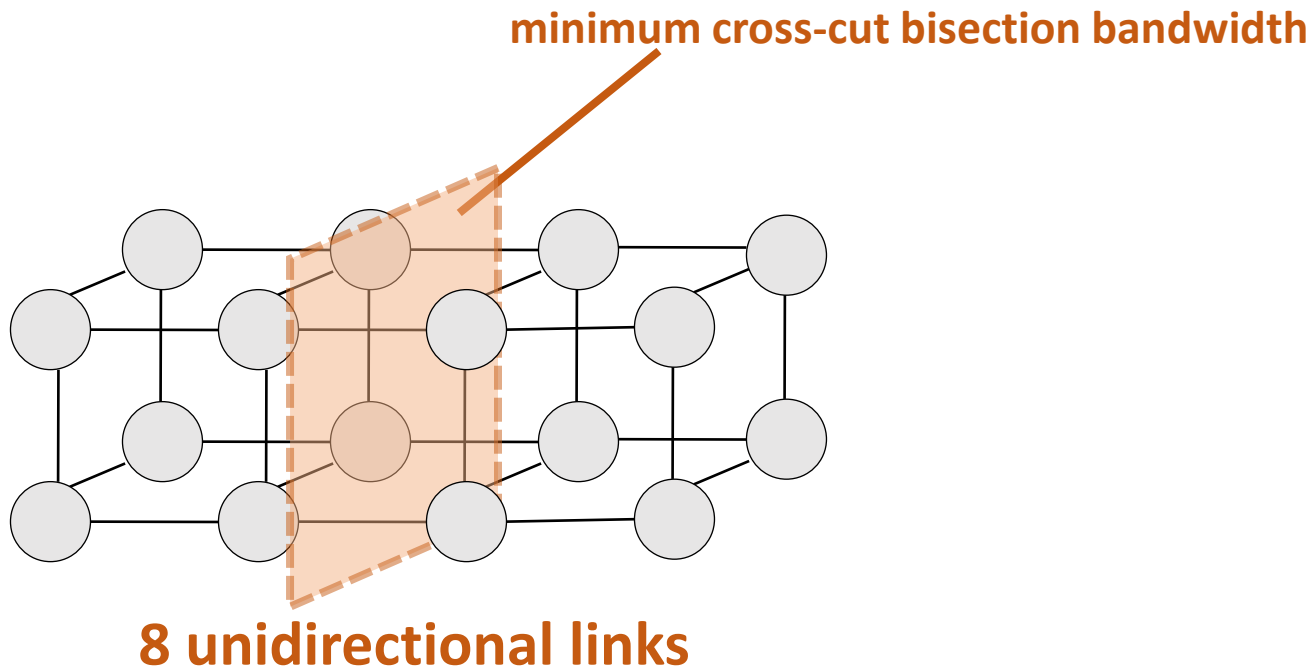
8 unidirectional links

One set of N/k bidirectional links crosses the bisection.

Direct Topologies – Mesh

Bisection Bandwidth:

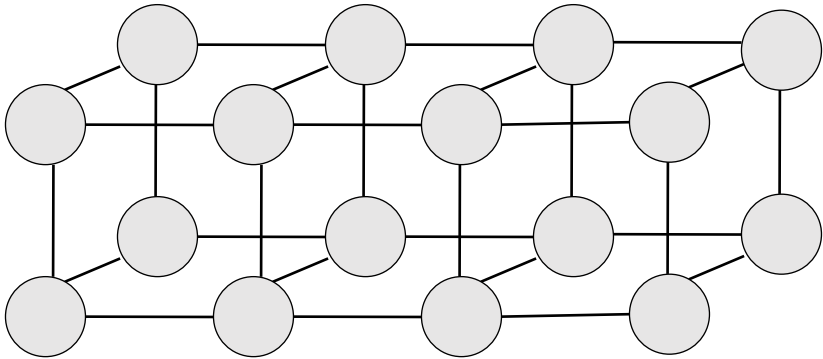
- Bisection bandwidth = $2N/k$ unidirectional links, largest k
- Half of torus bisection bandwidth



Direct Topologies – Mesh

Maximum Path Diversity:

- Assume shortest path routing

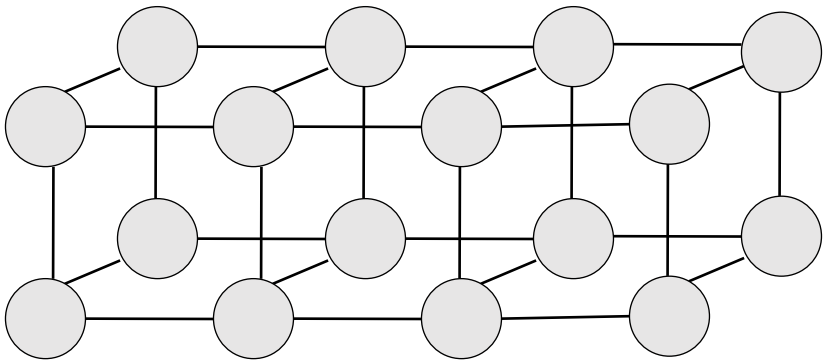


What's the maximum path diversity?

Direct Topologies – Mesh

Maximum Path Diversity:

- Assume shortest path routing

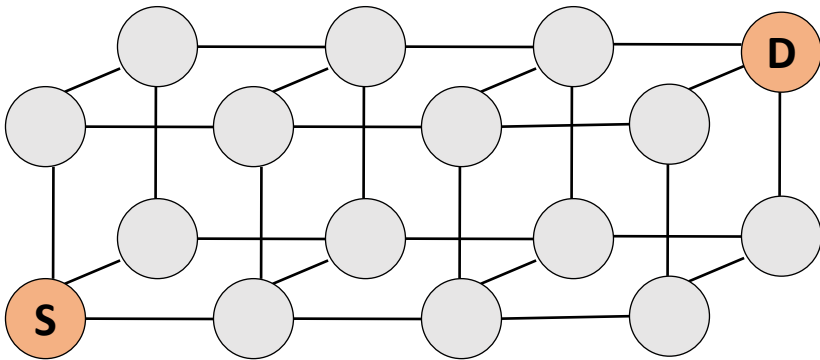


How many hops away is the furthest destination?

Direct Topologies – Mesh

Maximum Path Diversity:

- Assume shortest path routing



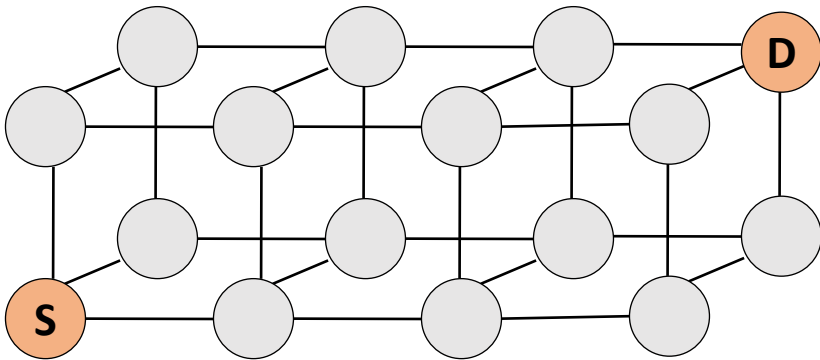
How many hops away is the furthest destination?

$k_1 - 1 + \dots + k_n - 1 = 5$ hops (i.e., network diameter)

Direct Topologies – Mesh

Maximum Path Diversity:

- Assume shortest path routing

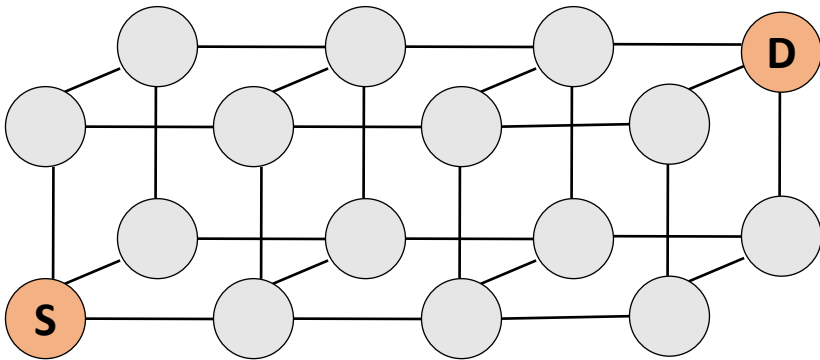


How many directions can we take?

Direct Topologies – Mesh

Maximum Path Diversity:

- Assume shortest path routing



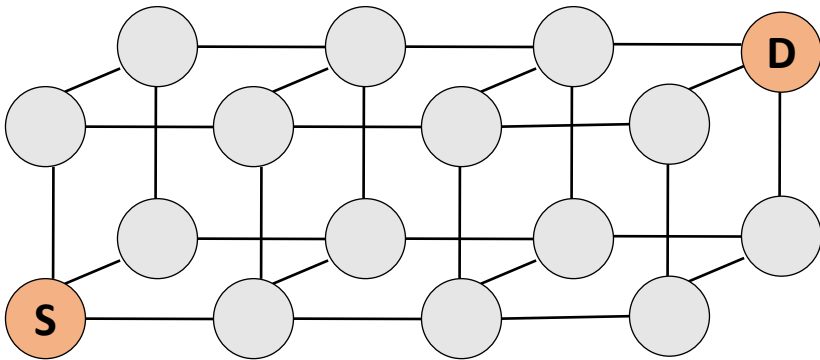
How many directions can we take?

1 direction

Direct Topologies – Mesh

Maximum Path Diversity:

- Assume shortest path routing



What's the maximum path diversity?

20 paths

$$\text{max path diversity} = \binom{(k_1 - 1) + \dots + (kn - 1)}{k_1 - 1, \dots, kn - 1} = 20$$

total hops

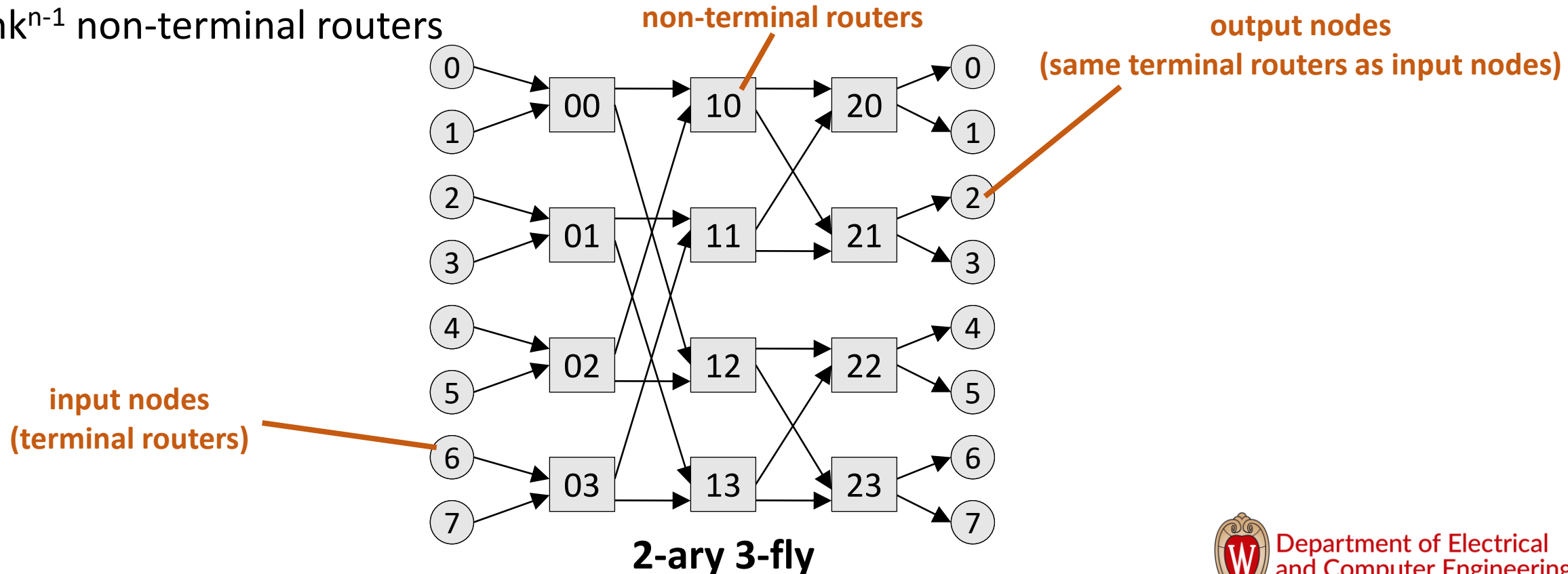
hops per dimension



Indirect Topologies – Butterfly

k-ary n-fly:

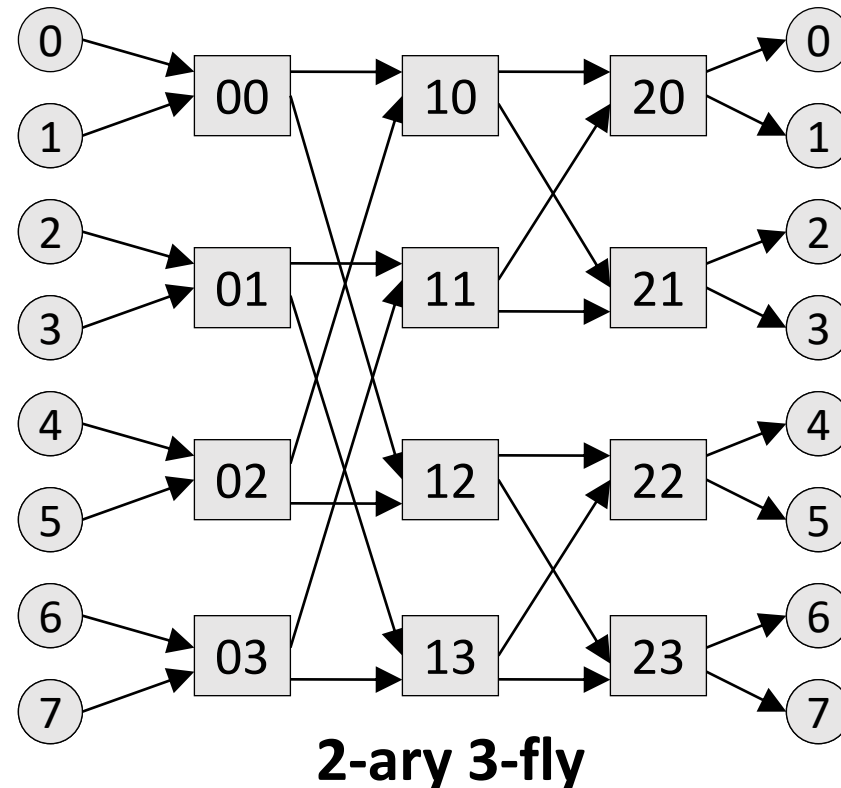
- n-stage network with router fan-in and fan-out of k
- k^n input/output nodes
- nk^{n-1} non-terminal routers



Indirect Topologies – Butterfly

Routing:

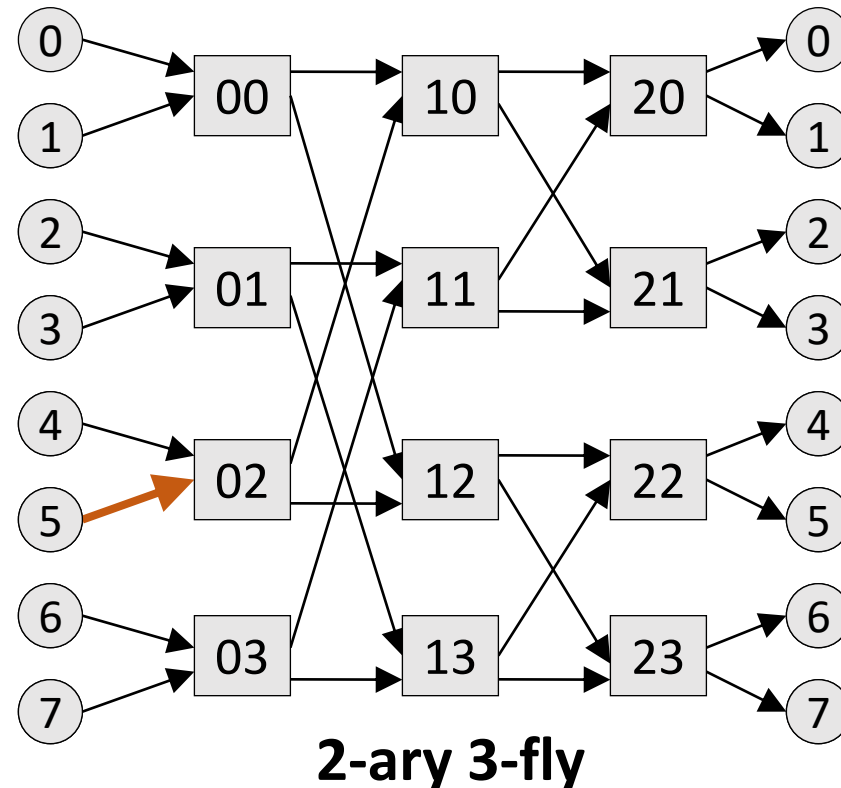
- Destination address used to directly route packet
- e.g., Packet from node 5 to node 2 (destination address 010)



Indirect Topologies – Butterfly

Routing:

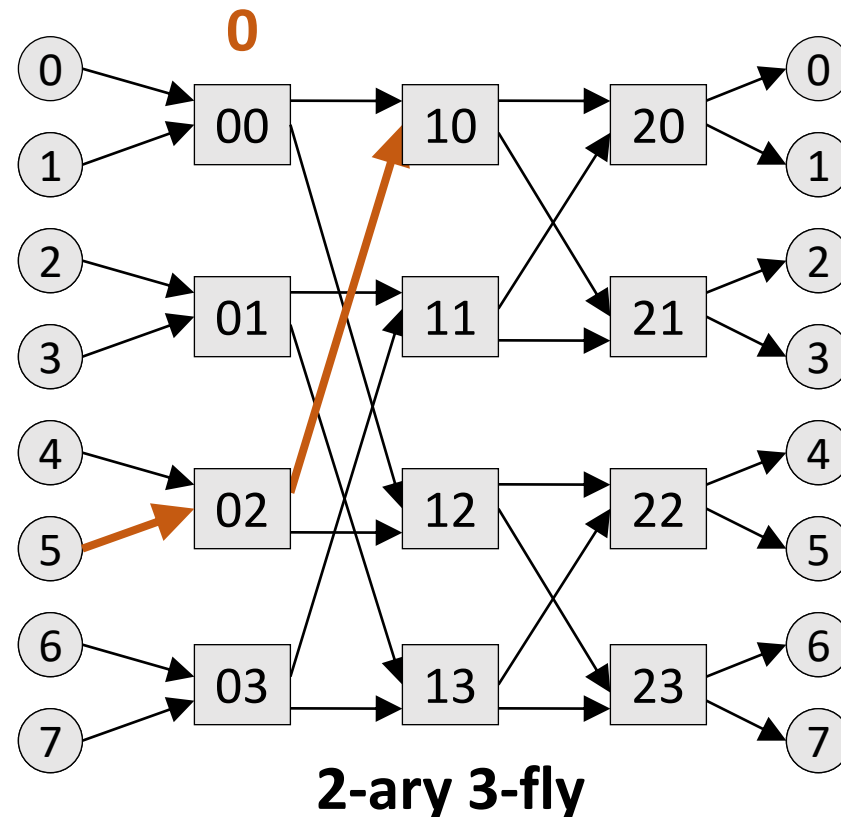
- Destination address used to directly route packet
- e.g., Packet from node 5 to node 2 (destination address 010)



Indirect Topologies – Butterfly

Routing:

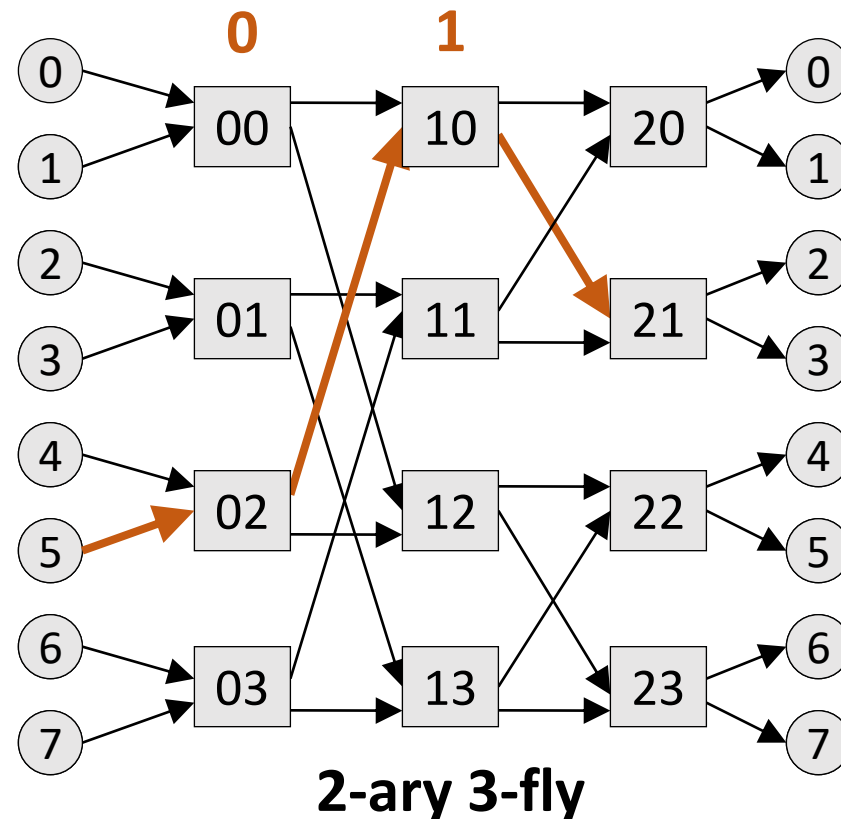
- Destination address used to directly route packet
- e.g., Packet from node 5 to node 2 (destination address 010)



Indirect Topologies – Butterfly

Routing:

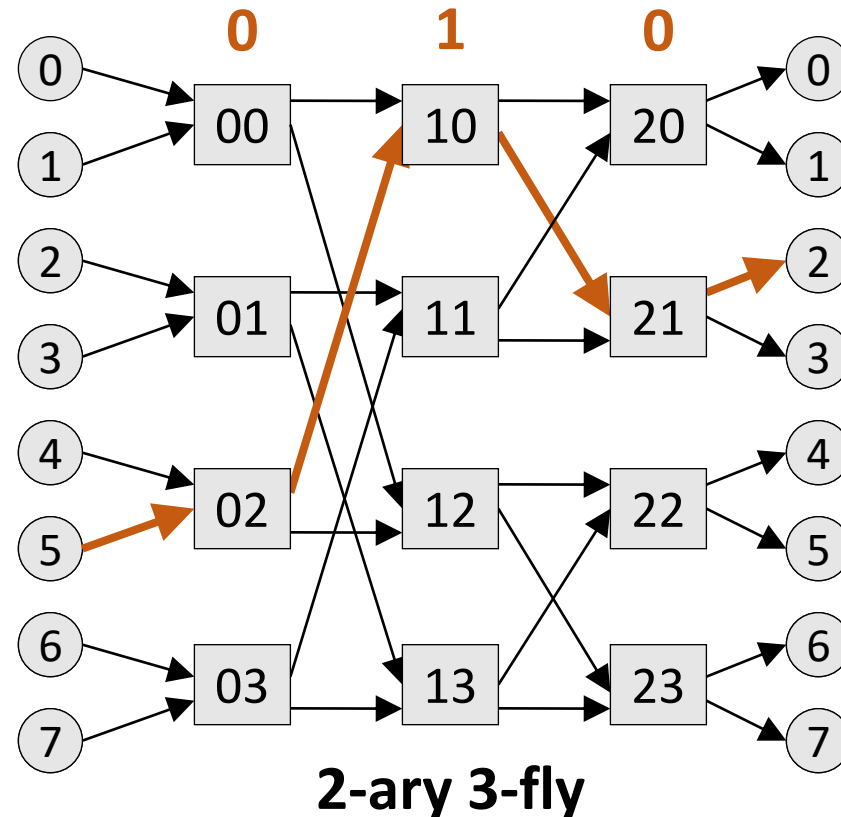
- Destination address used to directly route packet
- e.g., Packet from node 5 to node 2 (destination address 010)



Indirect Topologies – Butterfly

Routing:

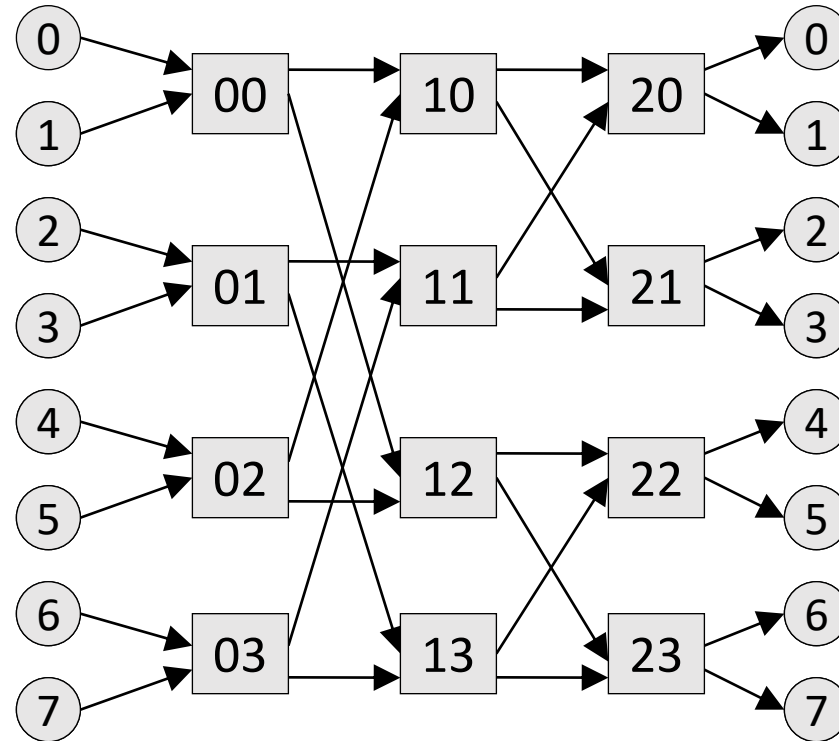
- Destination address used to directly route packet
- e.g., Packet from node 5 to node 2 (destination address 010)



Indirect Topologies – Butterfly

Degree:

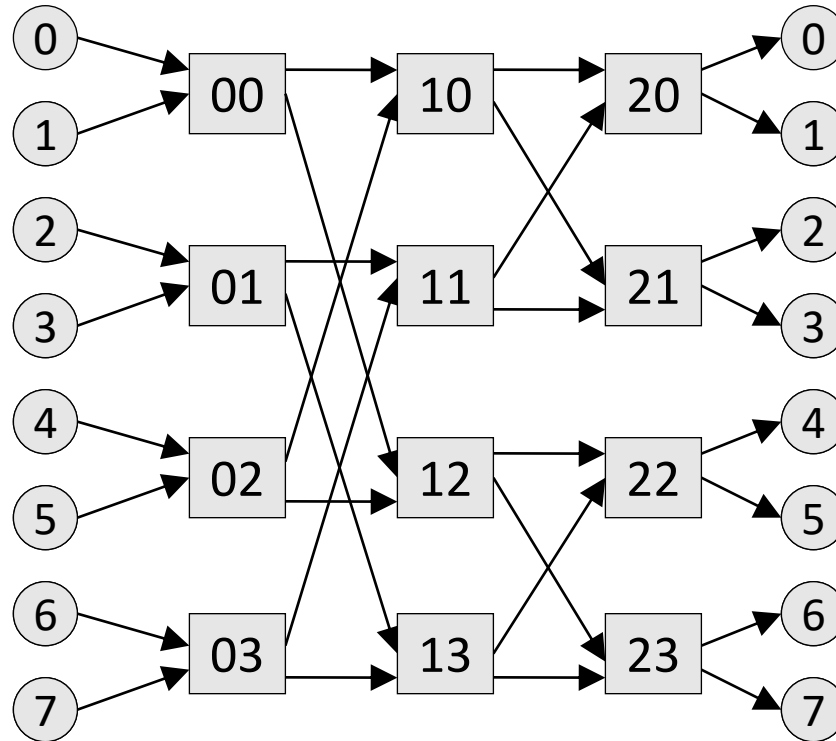
- Degree = k , for non-terminal routers



Indirect Topologies – Butterfly

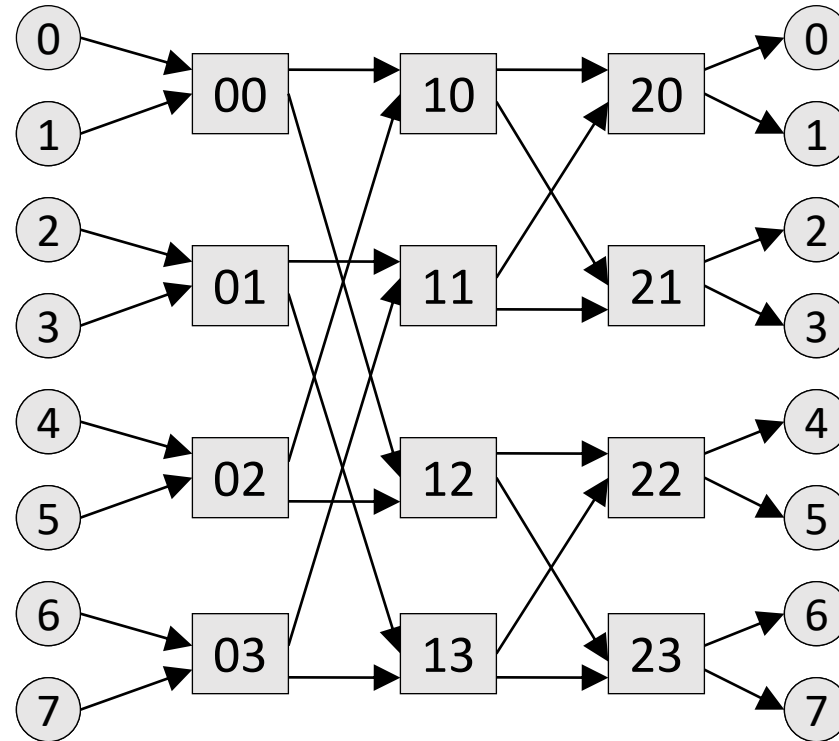
Network Diameter:

- Diameter = $n+1$ hops
- No locality; always the same hop count regardless of destination



Indirect Topologies – Butterfly

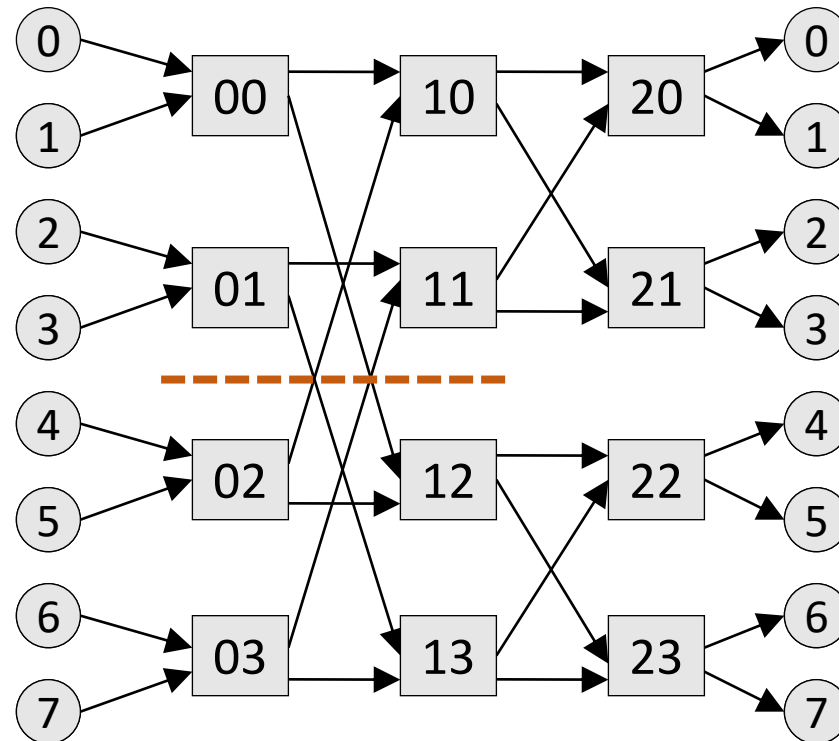
Bisection Bandwidth:



Indirect Topologies – Butterfly

Bisection Bandwidth:

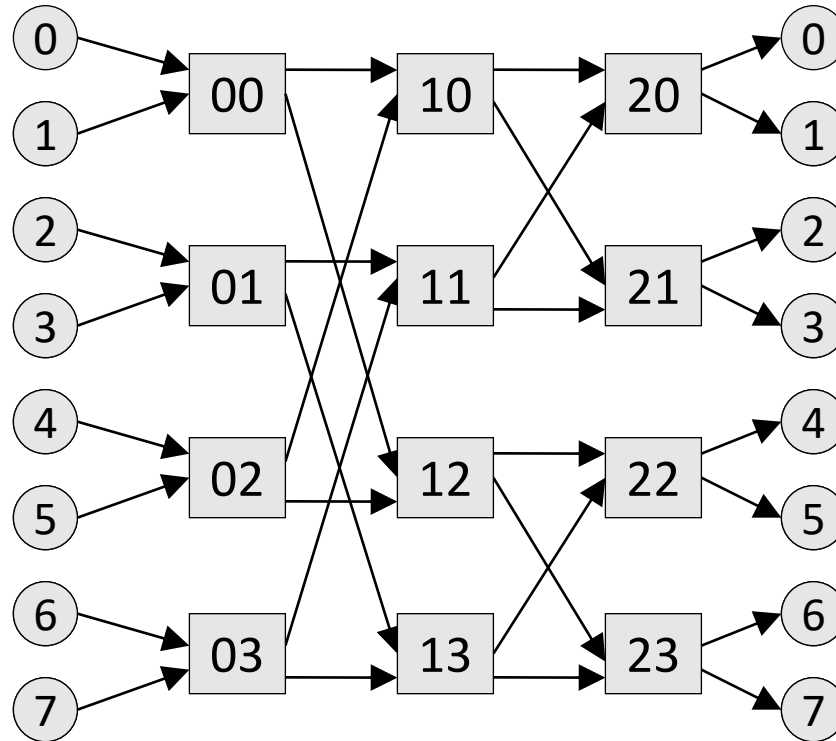
- $N/2 = 4$ unidirectional links



Indirect Topologies – Butterfly

Maximum (i.e., Highest Average) Channel Load:

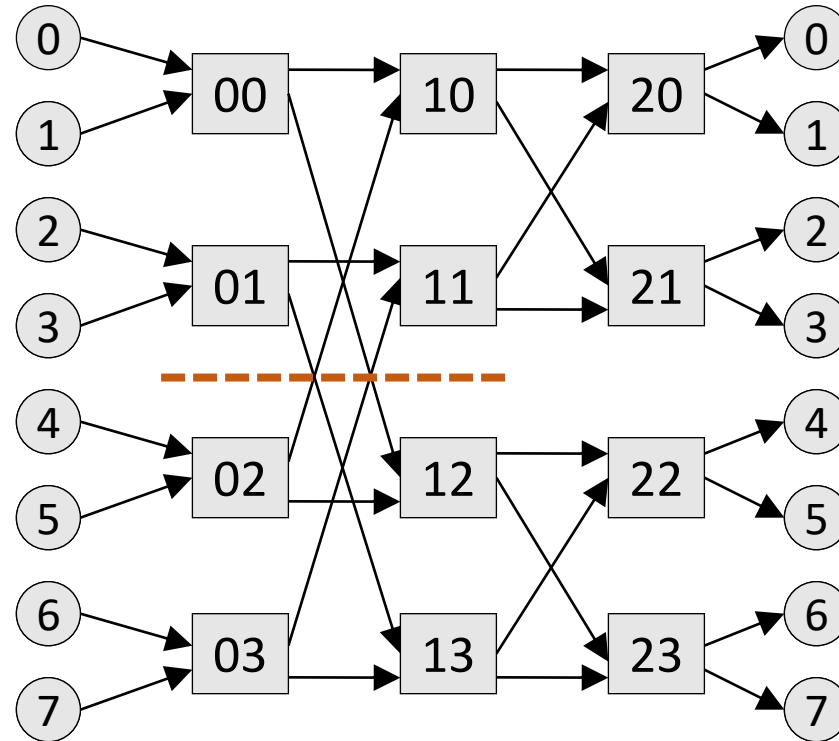
- Assume uniform random traffic
- Assume injection rate of 1 packet/node/cycle



Indirect Topologies – Butterfly

Maximum (i.e., Highest Average) Channel Load:

- Assume uniform random traffic
- Assume injection rate of 1 packet/node/cycle



Bisection = 4 links

Bisection traffic = $N/2$ packets/cycle

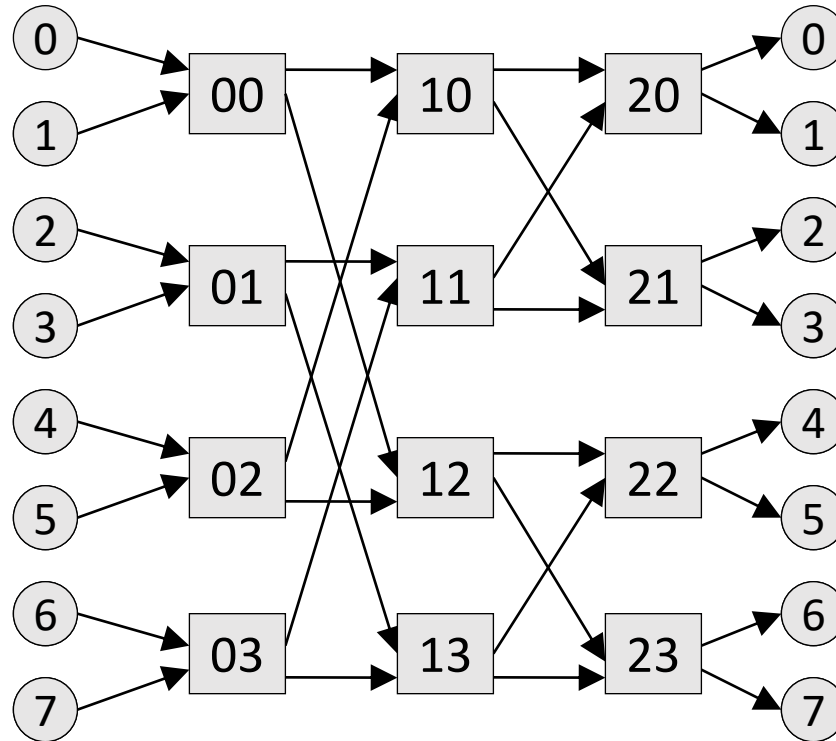
Channel load = 1 packet/cycle



Indirect Topologies – Butterfly

Maximum (i.e., Highest Average) Channel Load:

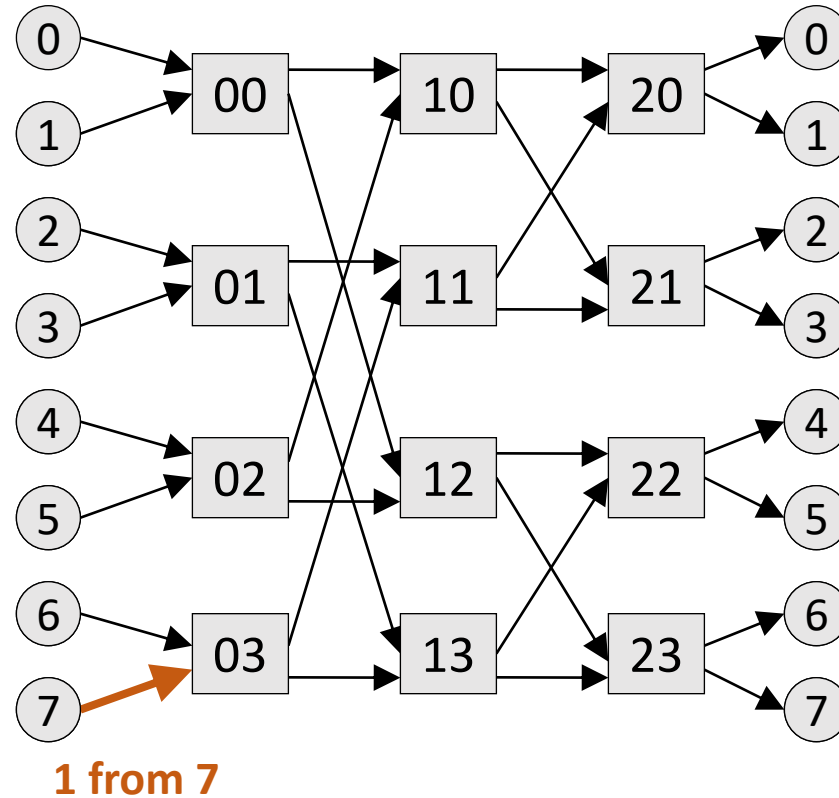
- Assume uniform random traffic
- Assume injection rate of 1 packet/node/cycle
- **Equally loaded channels**



Indirect Topologies – Butterfly

Maximum (i.e., Highest Average) Channel Load:

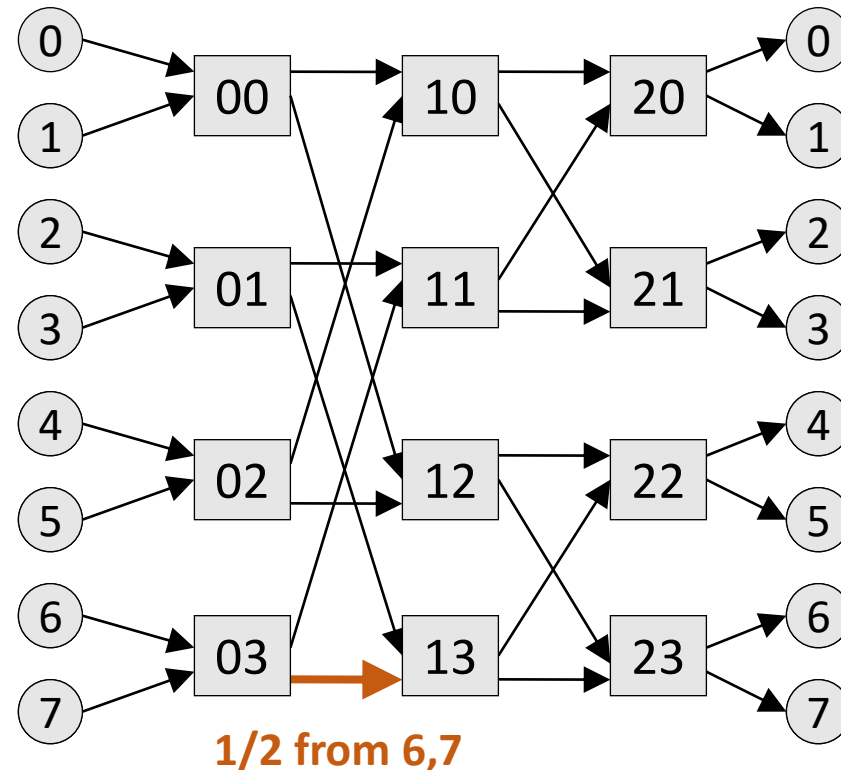
- Assume uniform random traffic
- Assume injection rate of 1 packet/node/cycle
- **Equally loaded channels**



Indirect Topologies – Butterfly

Maximum (i.e., Highest Average) Channel Load:

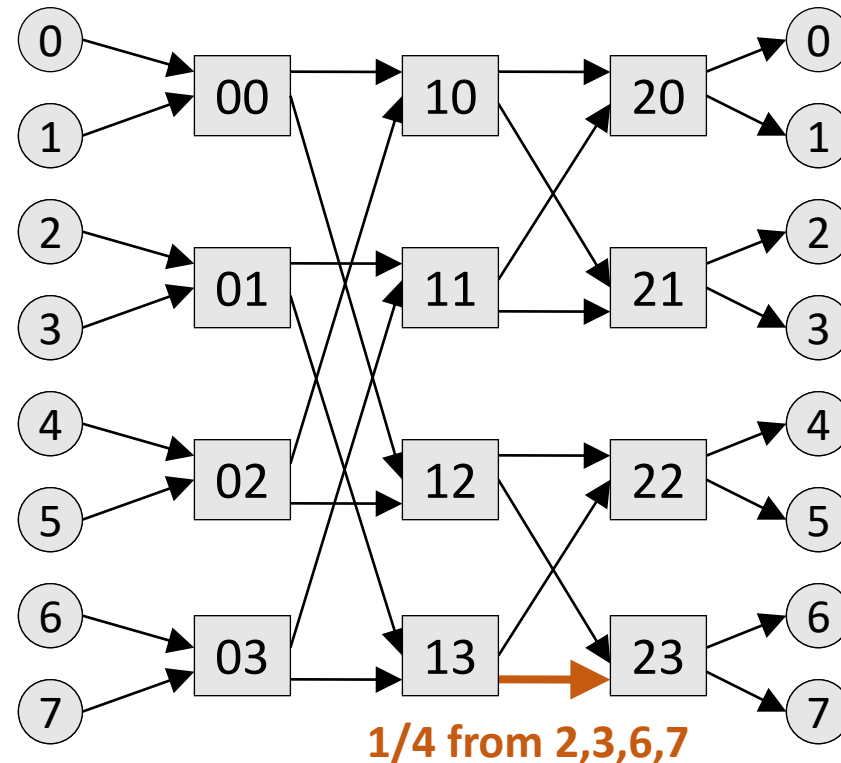
- Assume uniform random traffic
- Assume injection rate of 1 packet/node/cycle
- **Equally loaded channels**



Indirect Topologies – Butterfly

Maximum (i.e., Highest Average) Channel Load:

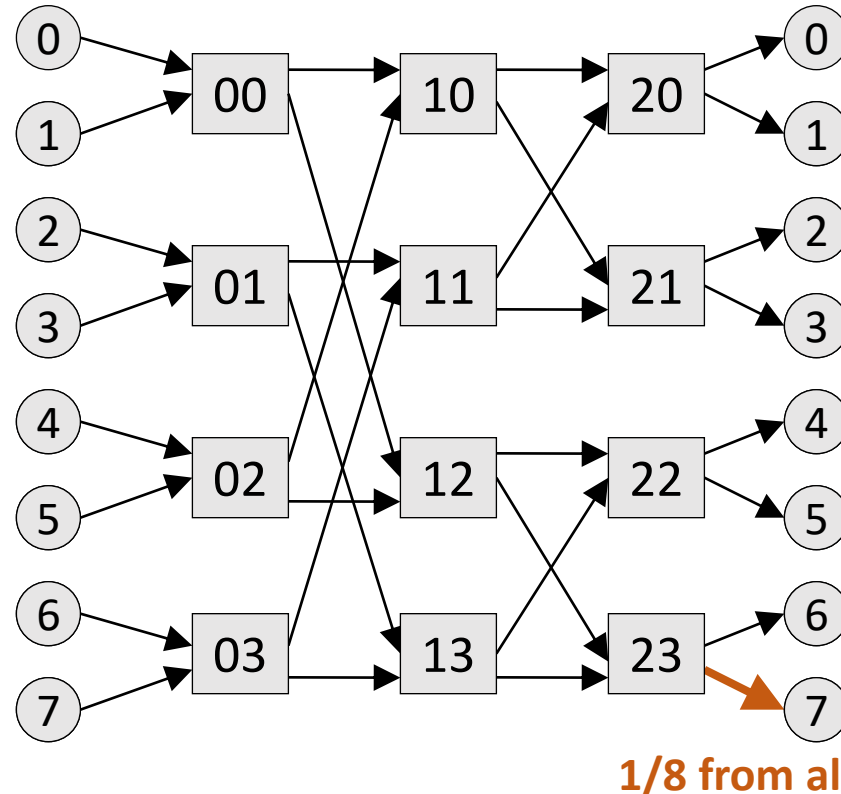
- Assume uniform random traffic
- Assume injection rate of 1 packet/node/cycle
- **Equally loaded channels**



Indirect Topologies – Butterfly

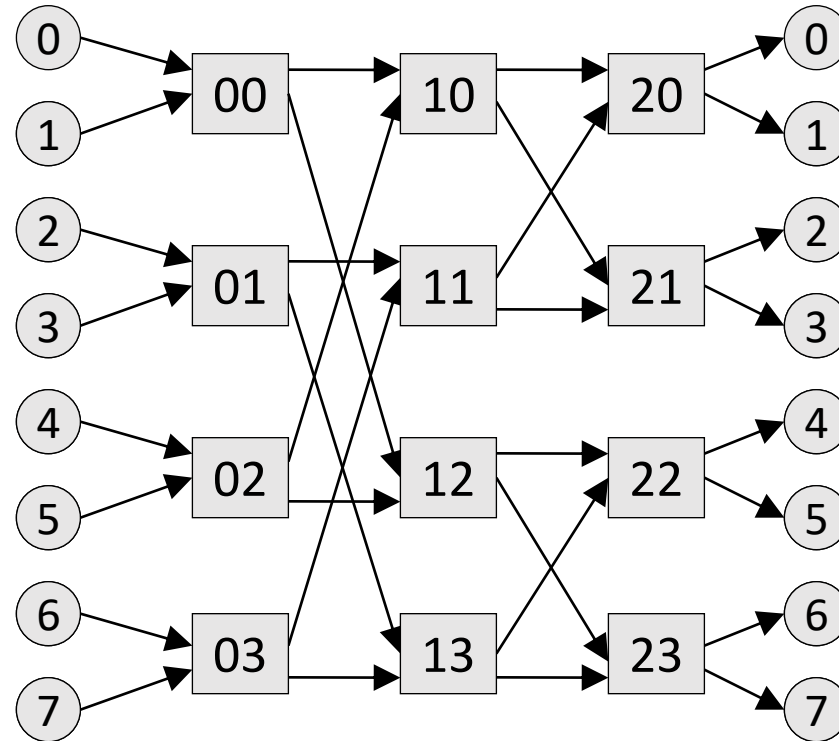
Maximum (i.e., Highest Average) Channel Load:

- Assume uniform random traffic
- Assume injection rate of 1 packet/node/cycle
- **Equally loaded channels**



Indirect Topologies – Butterfly

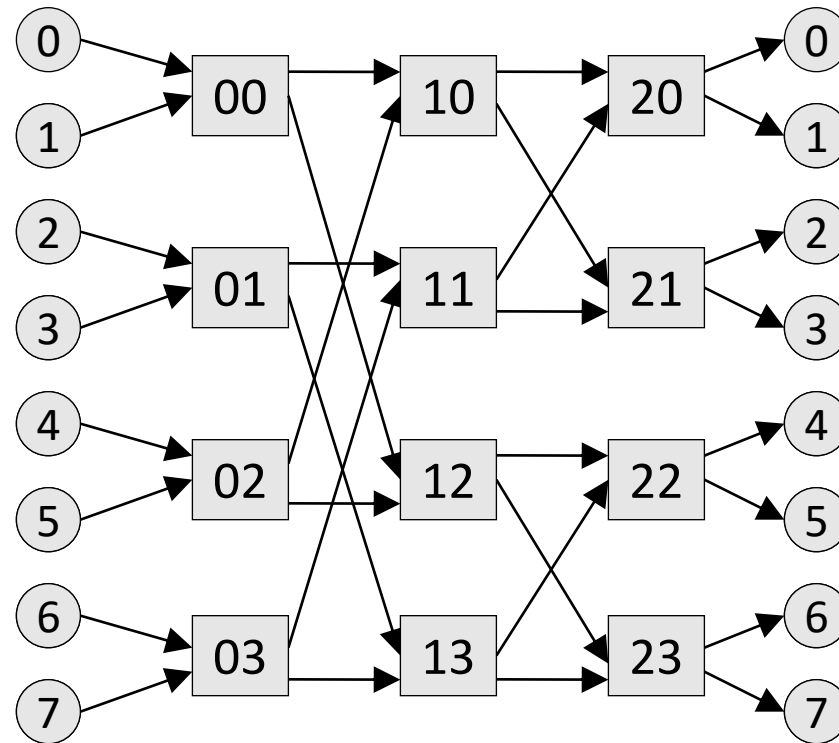
Path Diversity:



Indirect Topologies – Butterfly

Path Diversity:

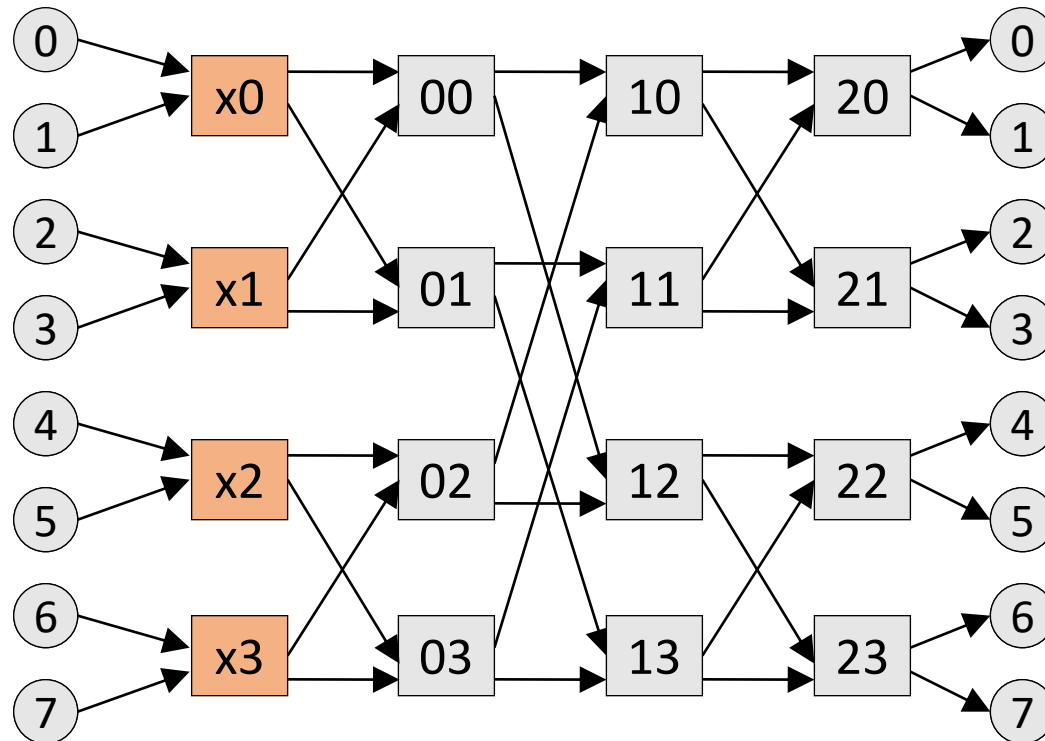
- 1 path



Indirect Topologies – Butterfly

Path Diversity:

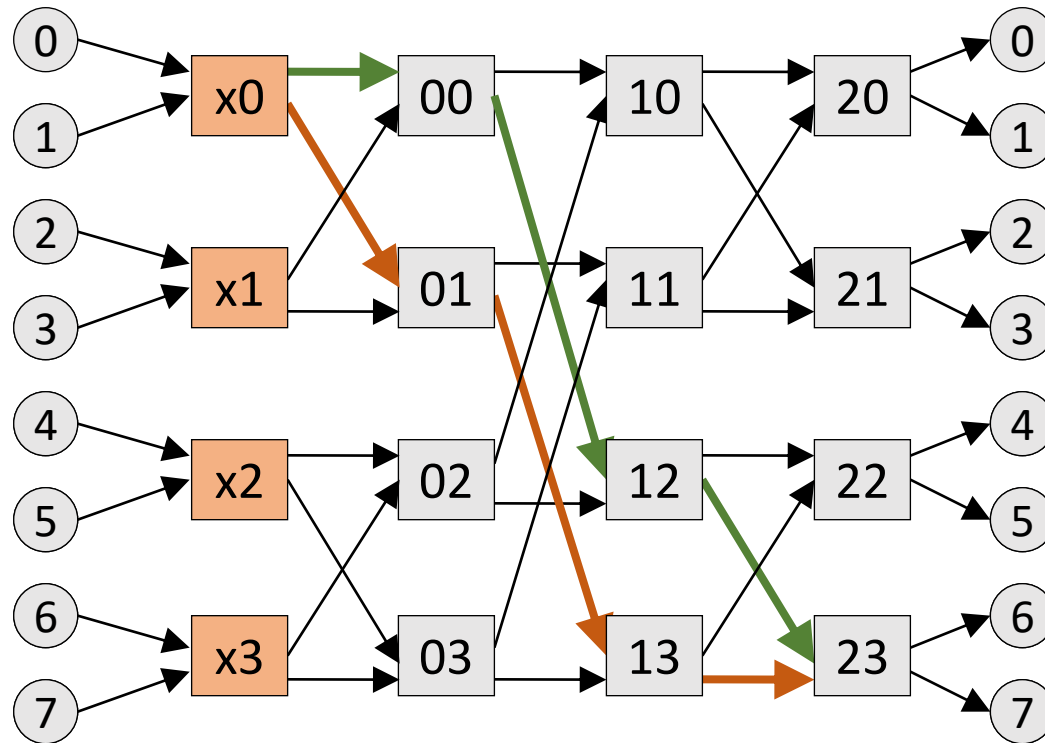
- 1 path
- Can add extra stages; though increases network diameter



Indirect Topologies – Butterfly

Path Diversity:

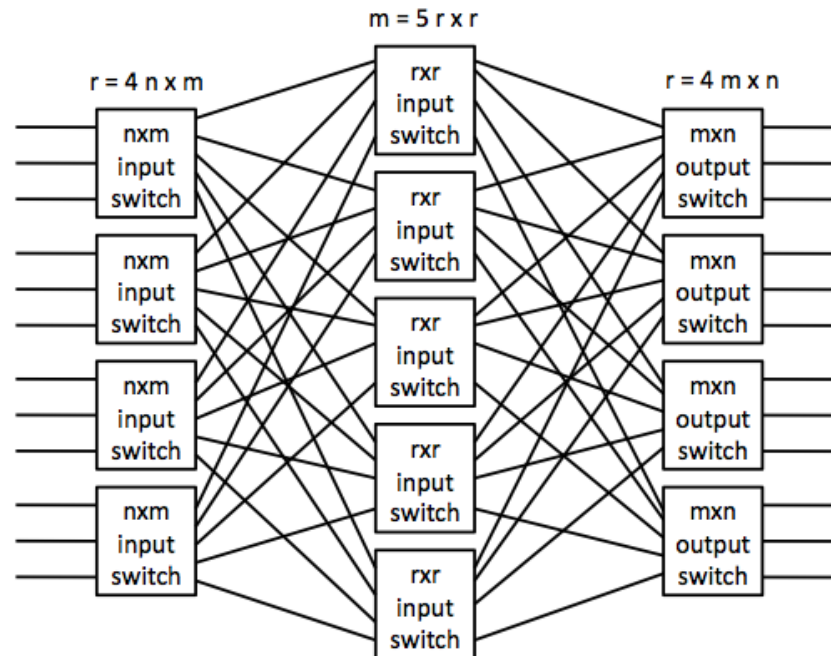
- 1 path
- Can add extra stages; though increases network diameter



Indirect Topologies – Clos Network

Clos Network:

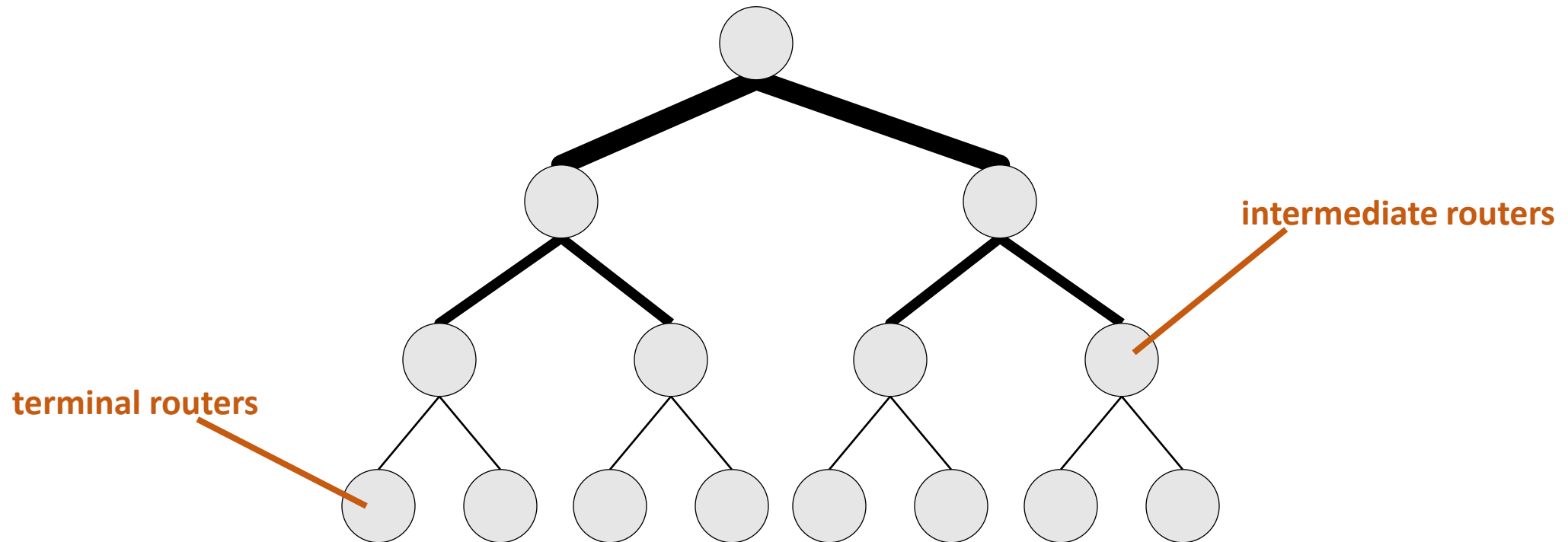
- 3-stage network where all input/output routers are connected to all middle routers
- Key goal: high path diversity
- Input node can select any middle router
- Can enable non-blocking routing algorithms



Indirect Topologies – Fat Tree

Fat Tree:

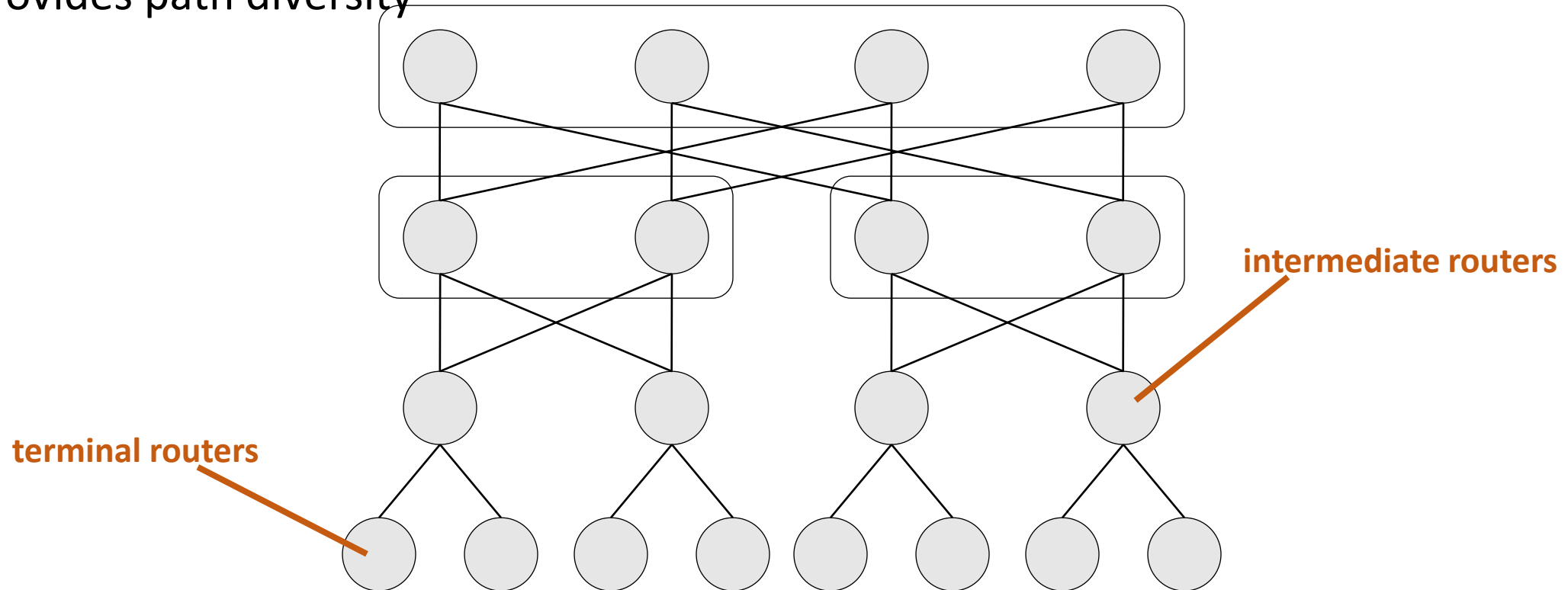
- Bandwidth remains constant at each level



Indirect Topologies – Fat Tree

Fat Tree:

- Bandwidth remains constant at each level
- Provides path diversity



Other Topologies

Flattened Butterfly:

- Convert butterfly to direct network

Swizzle Switch:

- Circuit-optimized crossbar

Multidrop Express Channels (MECS):

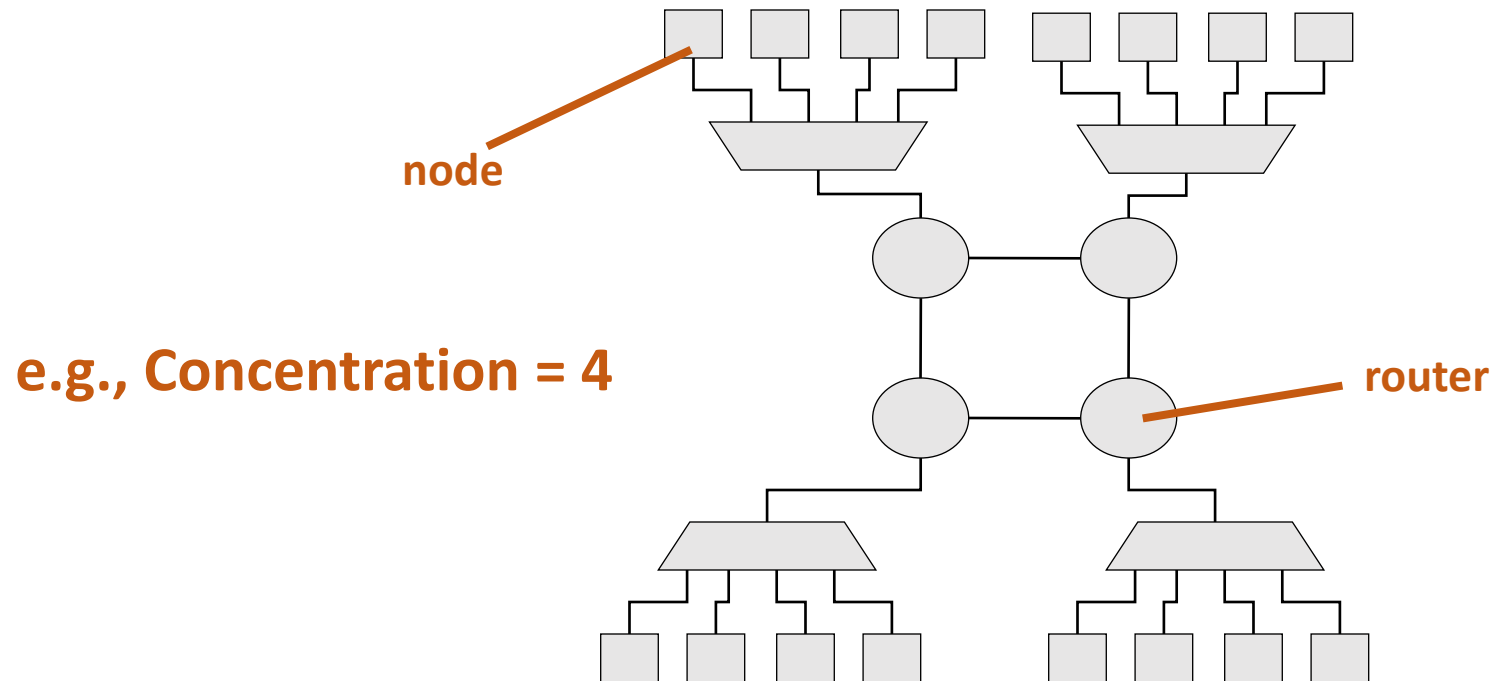
- Broadcast and multicast capabilities

Concentration

Concentration:

Number of nodes/devices per terminal router

- Can save area and power at the cost of complexity (must share injection bandwidth; bottleneck when bursty traffic)



Network-On-Chip

Key design decisions:

Topology

- Where to place routers/nodes and how to link them together

Routing

- Which set of links to traverse between source-destination pairs

Flow Control

- What granularity to move messages through links and routers

Router Microarchitecture

- How to pipeline and arbitrate incoming messages in routers

Routing

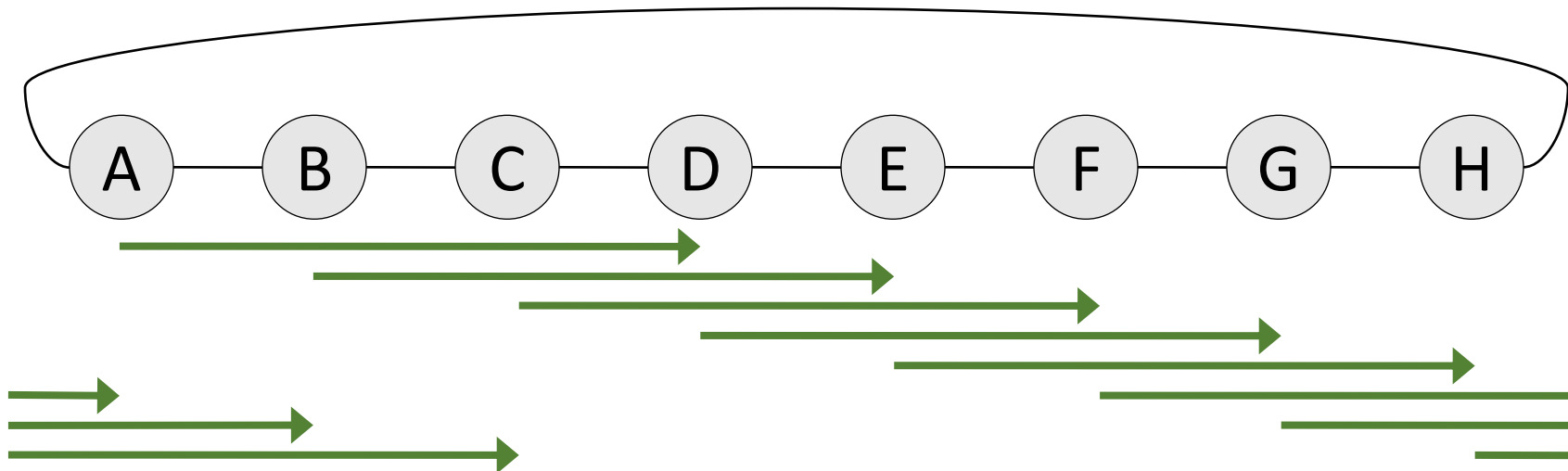
Routing algorithm determines path(s) from source to destination

- Analogous to GPS assistant
- Goal is to distribute traffic evenly among paths
- Avoid hot spots and contention
- More balance implies closer throughput to ideal
- But keep complexity in mind

Routing

e.g., Consider tornado traffic:

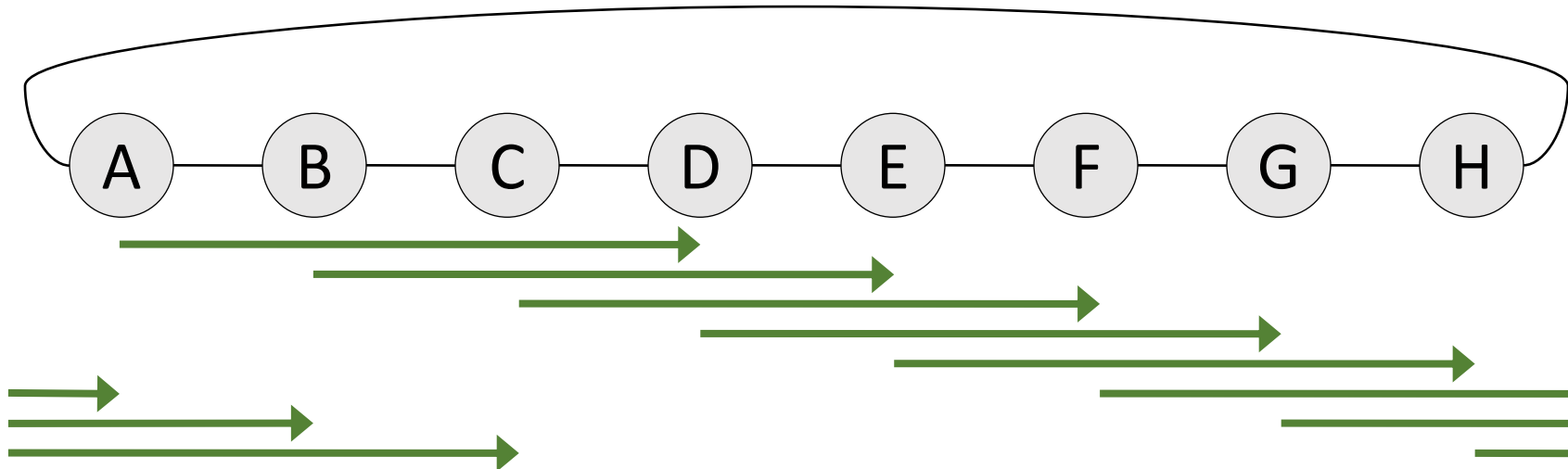
- Each router X sends one packet per cycle to router $(X+3)\%8$
- Thus injection rate of network is 8 packets/cycle
- Assume each unidirectional link can only handle one packet per cycle



Routing

Which routing algorithm gives best throughput?

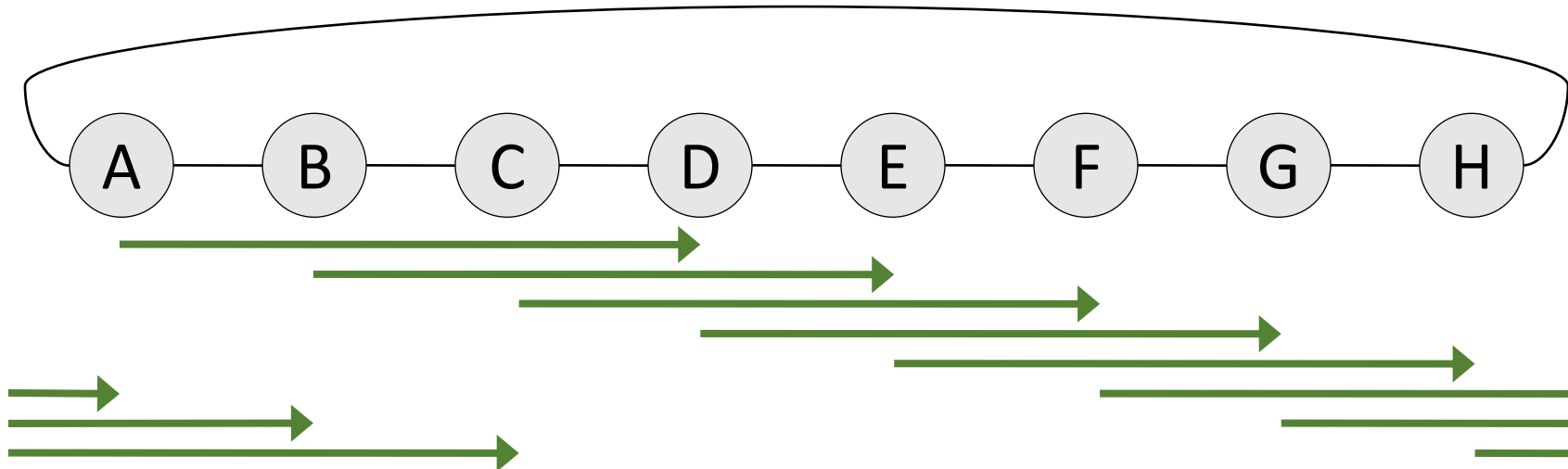
- Random: randomly pick initial direction
- Shortest Path: minimum hop count



Routing

Shortest Path:

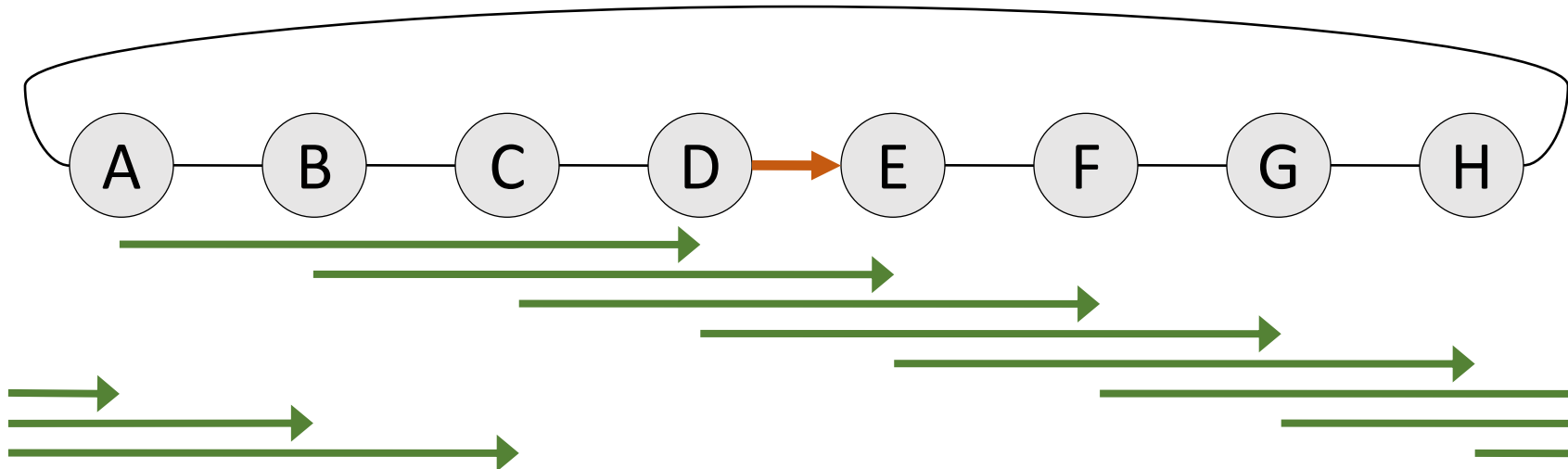
- All traffic moves counterclockwise (clockwise links unused)



Routing

Shortest Path:

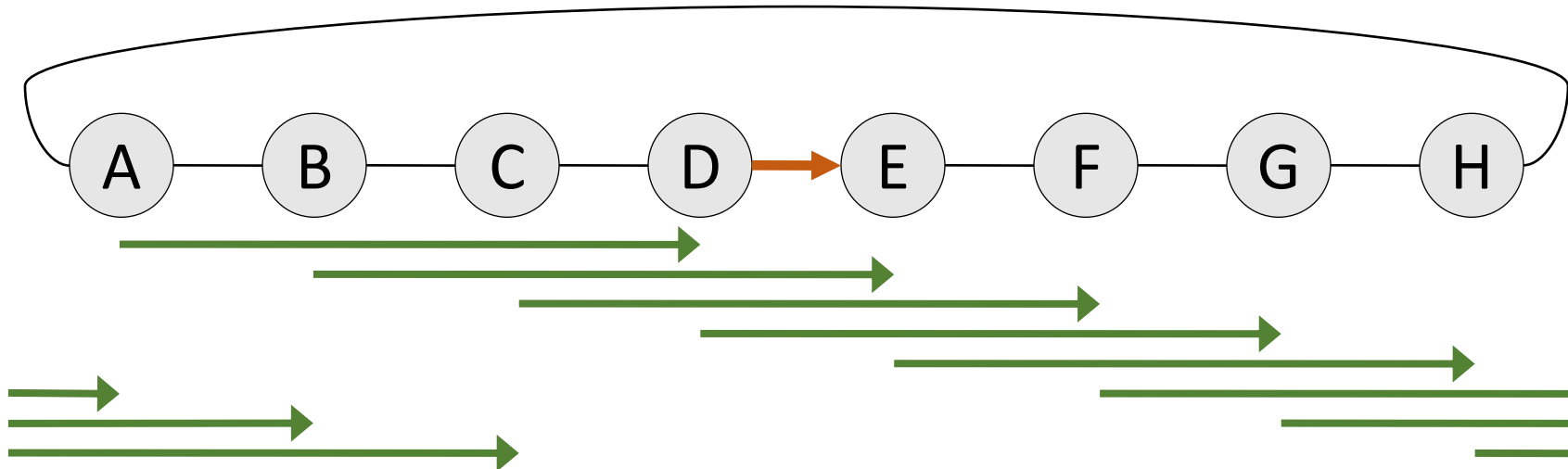
- All traffic moves counterclockwise (clockwise links unused)



Routing

Shortest Path:

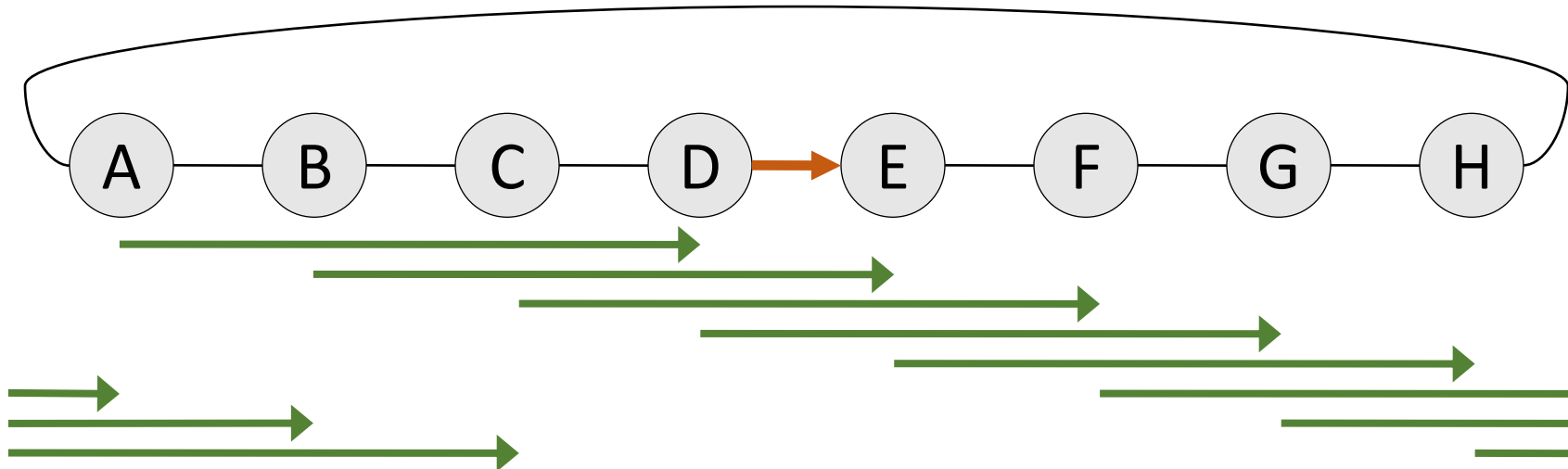
- All traffic moves counterclockwise (clockwise links unused)
- Counterclockwise channel load = 3 packets/cycle



Routing

Shortest Path:

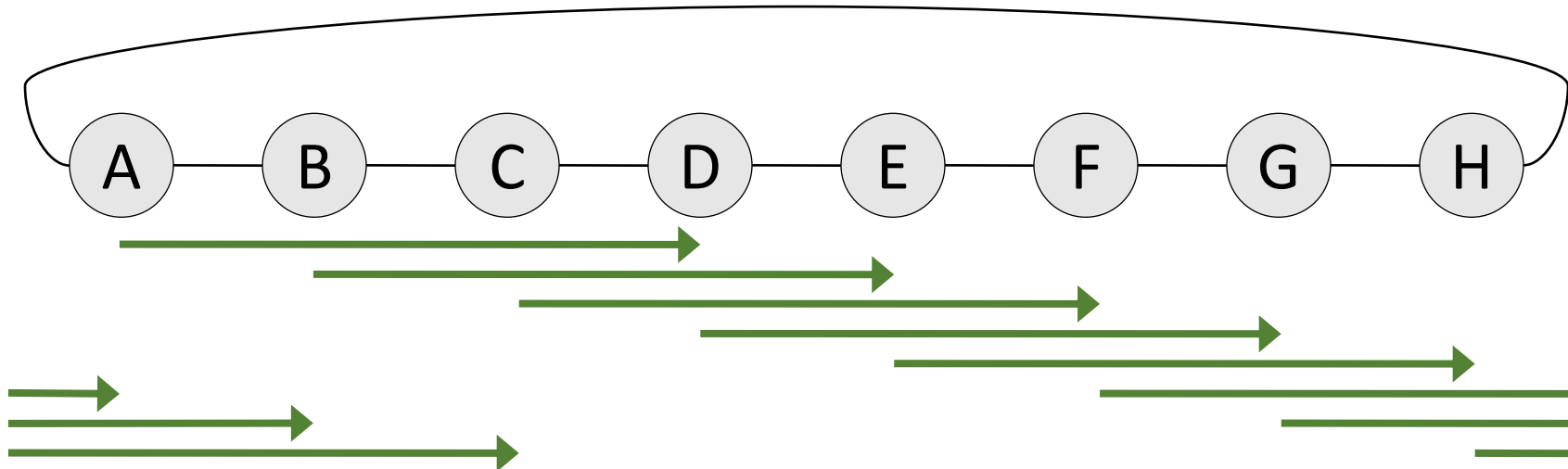
- All traffic moves counterclockwise (clockwise links unused)
- Counterclockwise channel load = 3 packets/cycle
- Therefore, network throughput is 1/3 of injection (2.67 packets/cycle)



Routing

Random:

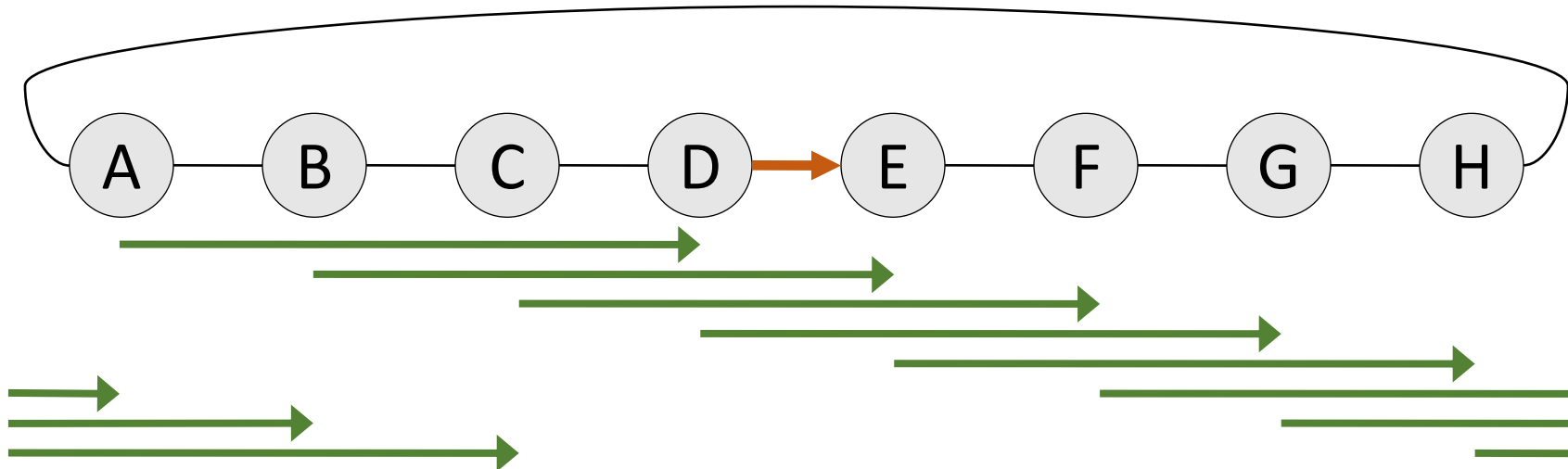
Which links are most congested?



Routing

Random:

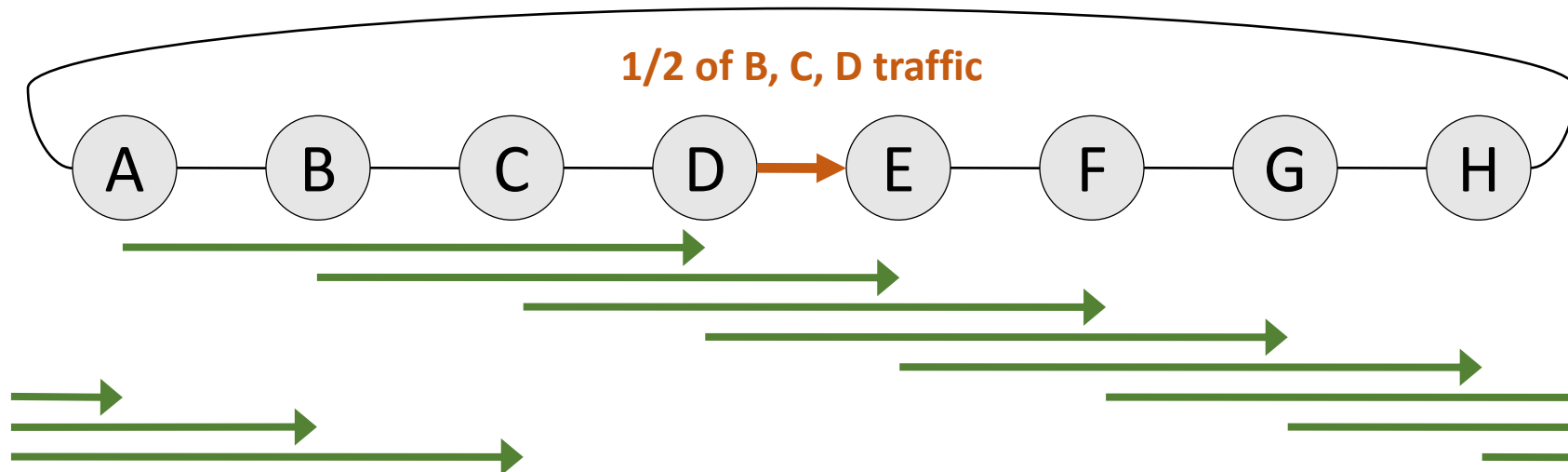
Which links are most congested?



Routing

Random:

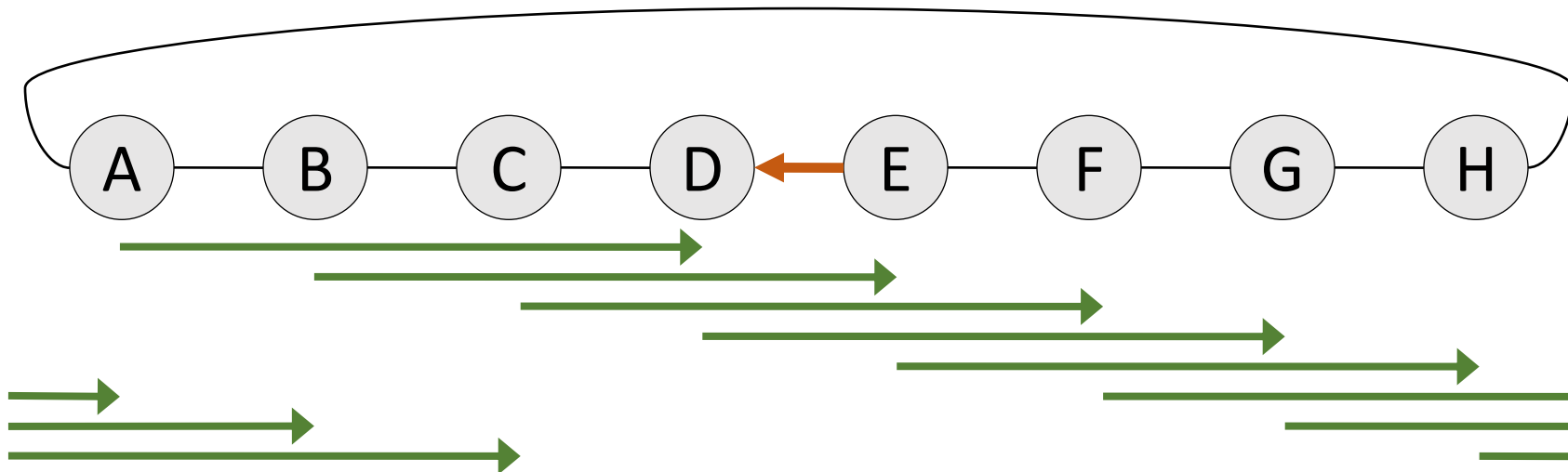
Which links are most congested?



Routing

Random:

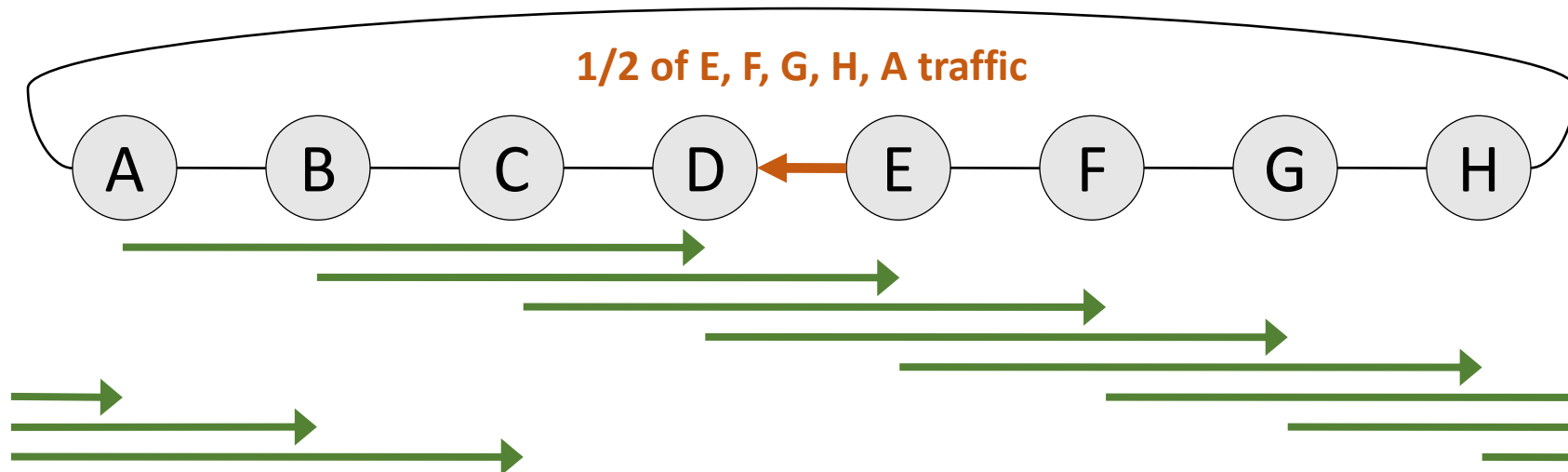
Which links are most congested?



Routing

Random:

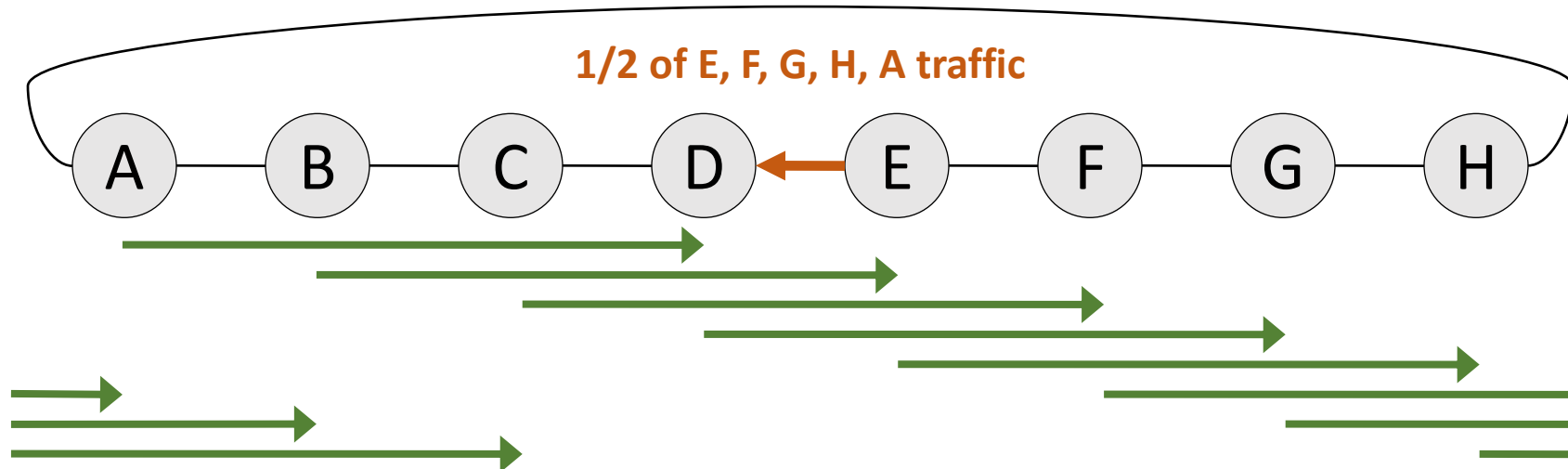
Which links are most congested?



Routing

Random:

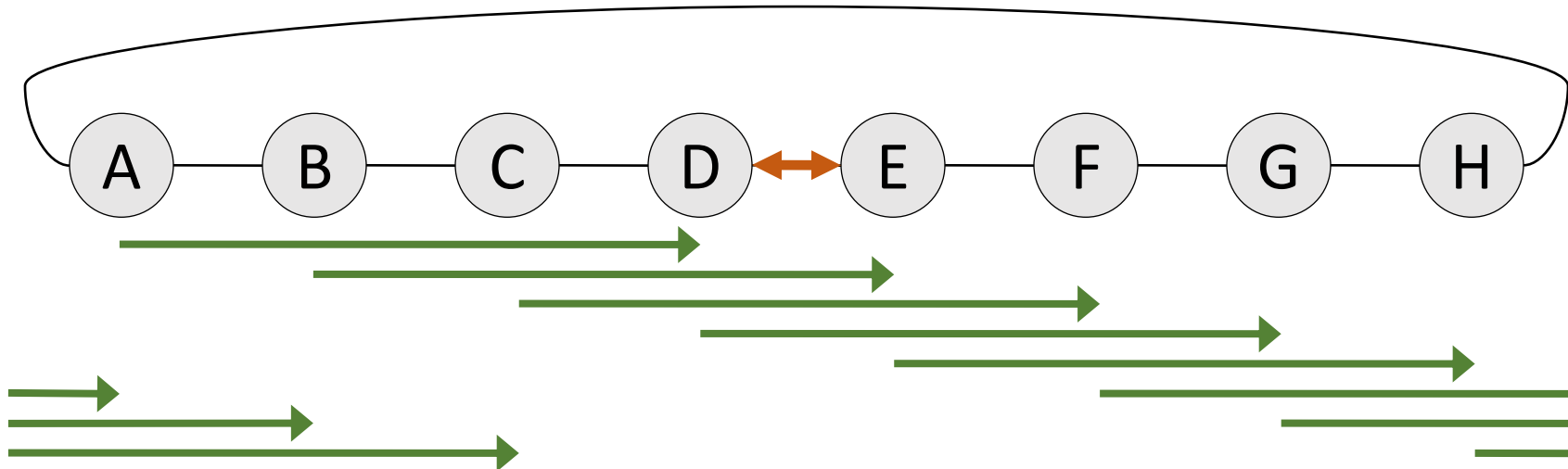
- Clockwise links become bottleneck since more in-flight packets
- Clockwise channel load = $5/2$ packets/cycle
- Therefore, network throughput is $2/5$ of injection (3.2 packets/cycle)



Routing

Adaptive:

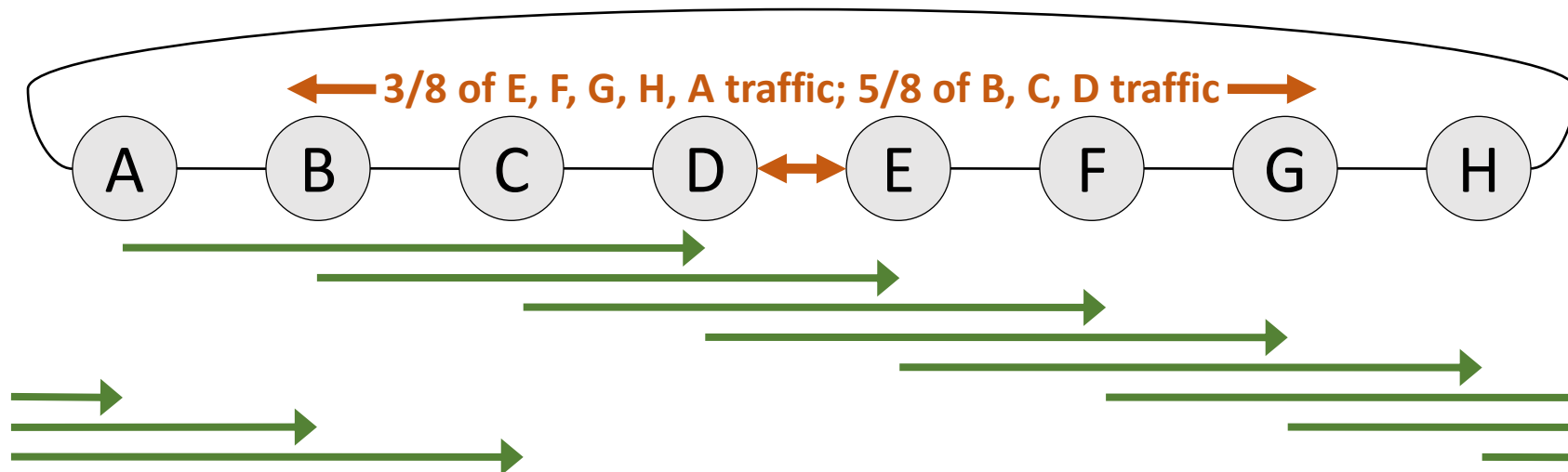
- Perfect load balancing



Routing

Adaptive:

- Perfect load balancing: $5/8$ traffic counterclockwise, $3/8$ traffic clockwise
- Channel load = $15/8$ packets/cycle
- Therefore, network throughput is $8/15$ of injection (4.27 packets/cycle)



Routing – Design Decisions

Types of routing algorithms:

- Deterministic, oblivious, adaptive?

Adaptivity:

- Local information, global information?
- Minimal, non-minimal?

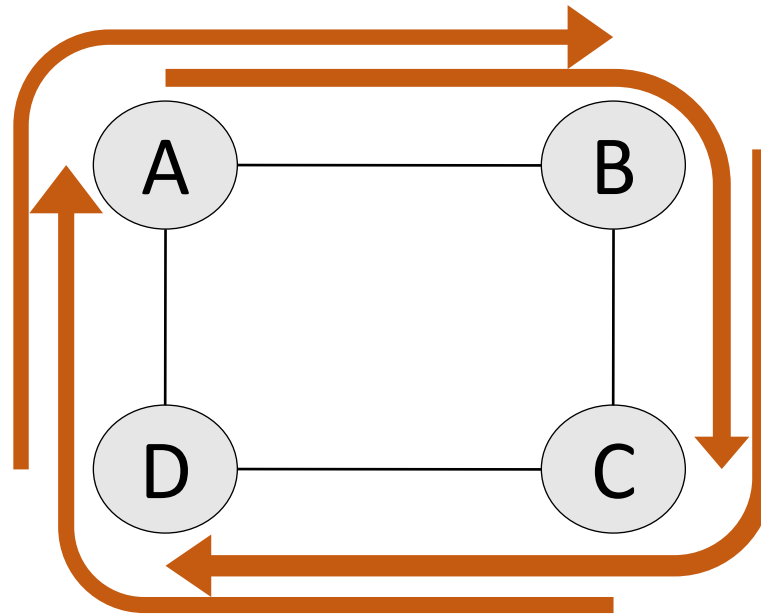
Implementation:

- Source routing, node routing?
- Routing table, routing circuit?

Routing – Deadlock

Deadlock:

- Network resource cycle
- Each packet is both occupying a link and waiting for a link
- Without routing restrictions, deadlocks can occur



Deterministic Routing

All packets from source S to destination D traverse the same path

Disadvantages:

- Eliminates any path diversity provided by topology
- Poor load balancing on-the-fly (cannot reroute around congestion)

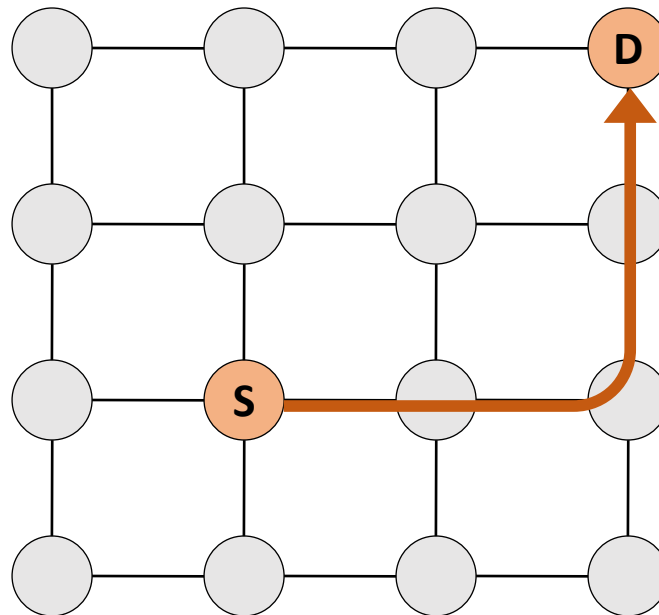
Advantages:

- Inexpensive to implement
- Simple to ensure deadlock-freedom

Deterministic Routing – Dimension-Order

Dimension-Order Routing (DOR or X-Y Routing):

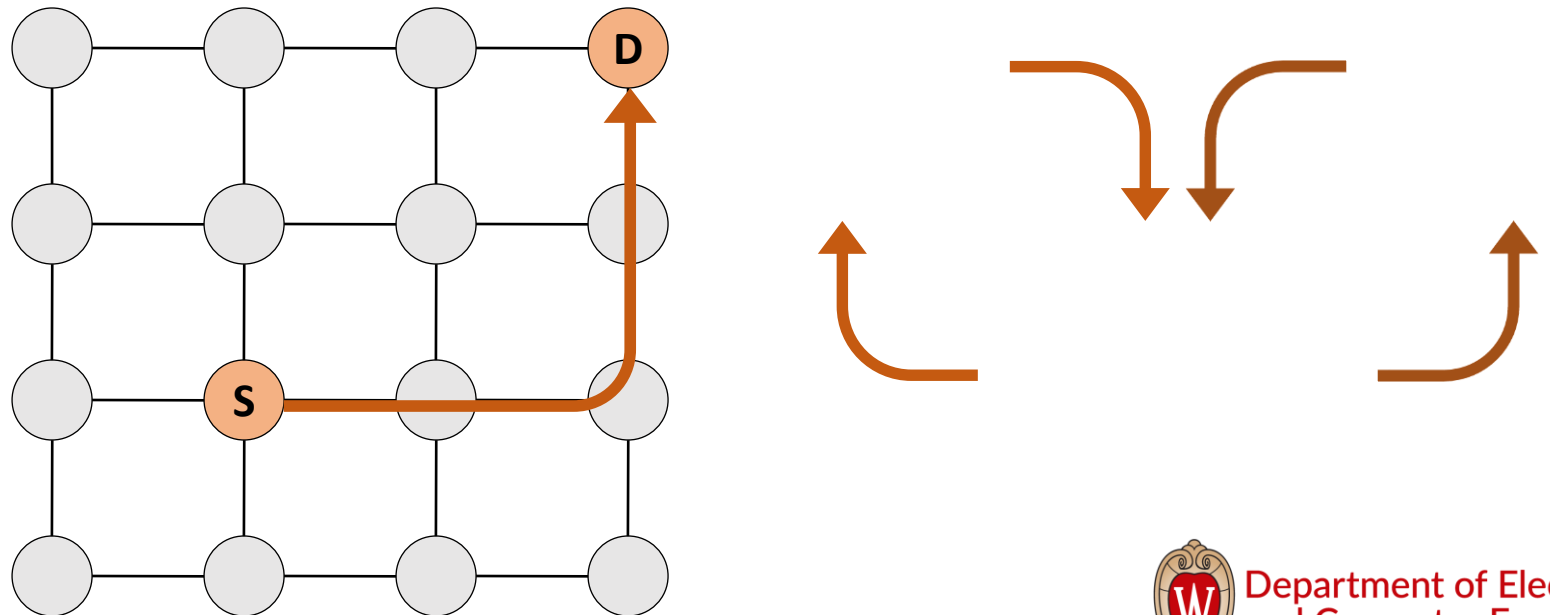
- Traverse the network dimension by dimension, at a predefined order
- Can only turn to the next dimension (Y) after finished the current dimension (X)
- Cannot traverse any previously finished dimension



Deterministic Routing – Dimension-Order

Dimension-Order Routing (DOR or X-Y Routing):

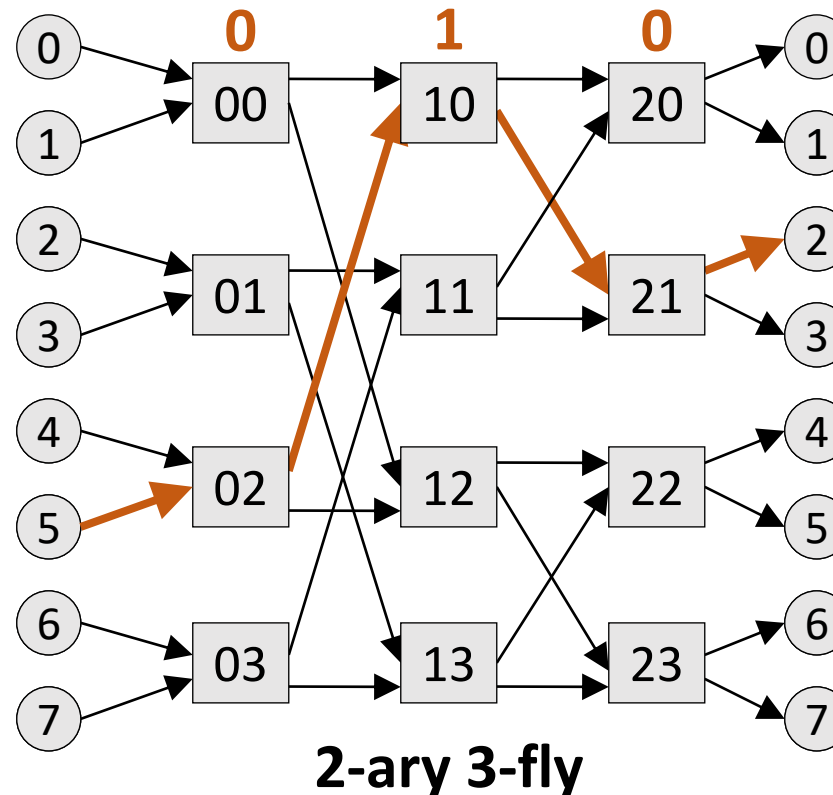
- Traverse the network dimension by dimension, at a predefined order
- Can only turn to the next dimension (Y) after finished the current dimension (X)
- Cannot traverse any previously finished dimension
- Ensures deadlock-freedom by allowing only some turns:



Deterministic Routing – Destination-Tag

Destination-Tag Routing:

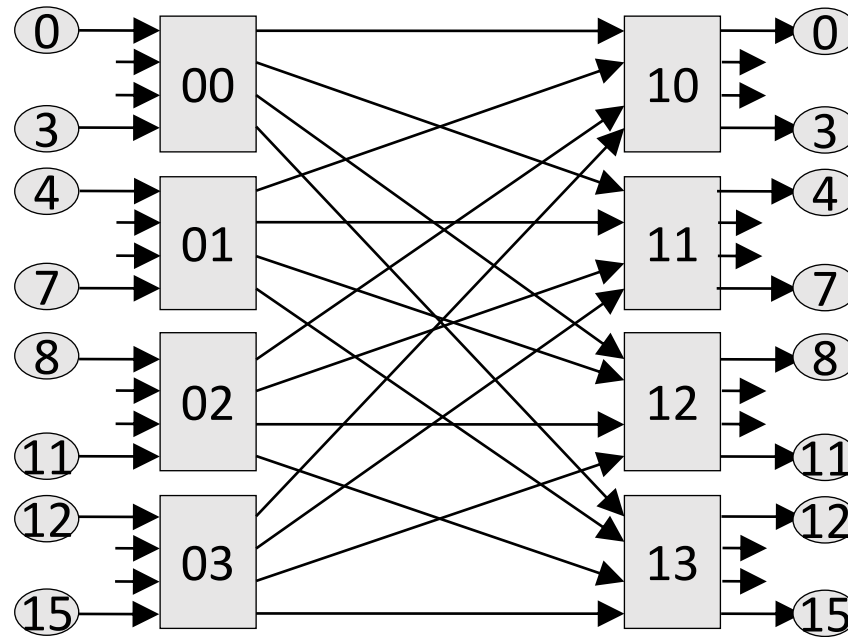
- Destination address directly routes packet in butterfly networks
- Destination address interpreted as n-digit radix-k number
- Each digit (starting from most-significant) selects the output port at each stage



Deterministic Routing – Destination-Tag

Destination-Tag Routing:

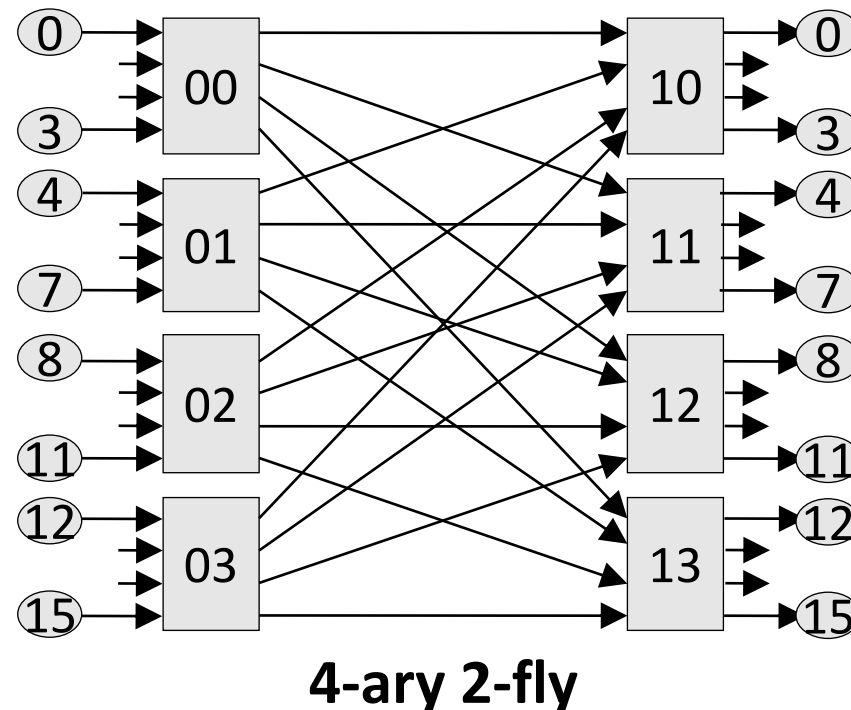
- Destination address directly routes packet in butterfly networks
- Destination address interpreted as n-digit radix-k number
- Each digit (starting from most-significant) selects the output port at each stage



Deterministic Routing – Destination-Tag

Destination-Tag Routing:

- Destination address directly routes packet in butterfly networks
- Destination address interpreted as n-digit radix-k number
- Each digit (starting from most-significant) selects the output port at each stage

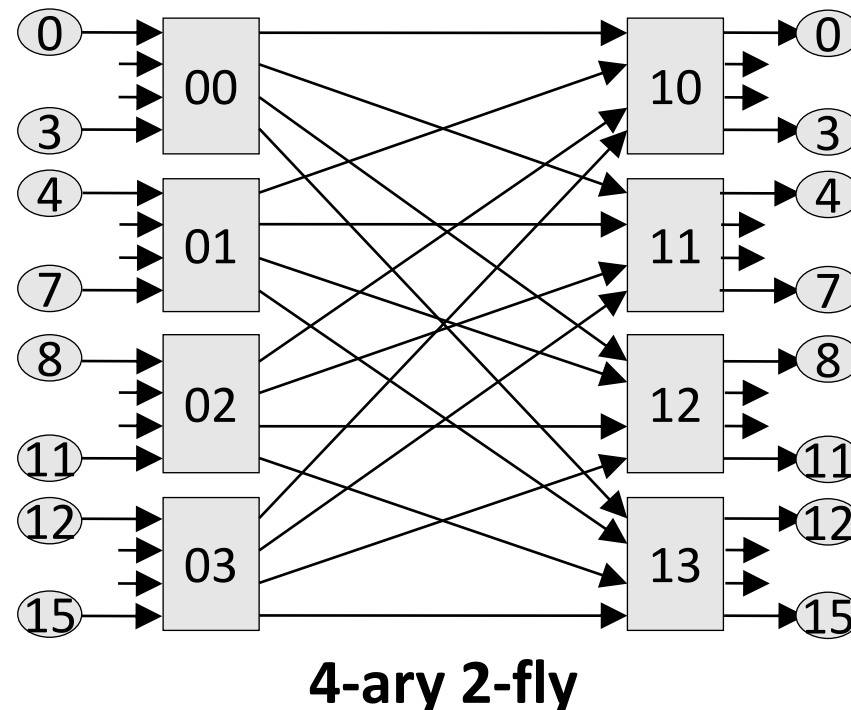


Deterministic Routing – Destination-Tag

Destination-Tag Routing:

- Destination address directly routes packet in butterfly networks
- Destination address interpreted as n-digit radix-k number
- Each digit (starting from most-significant) selects the output port at each stage

e.g., Node 7 to Node 11:



Deterministic Routing – Destination-Tag

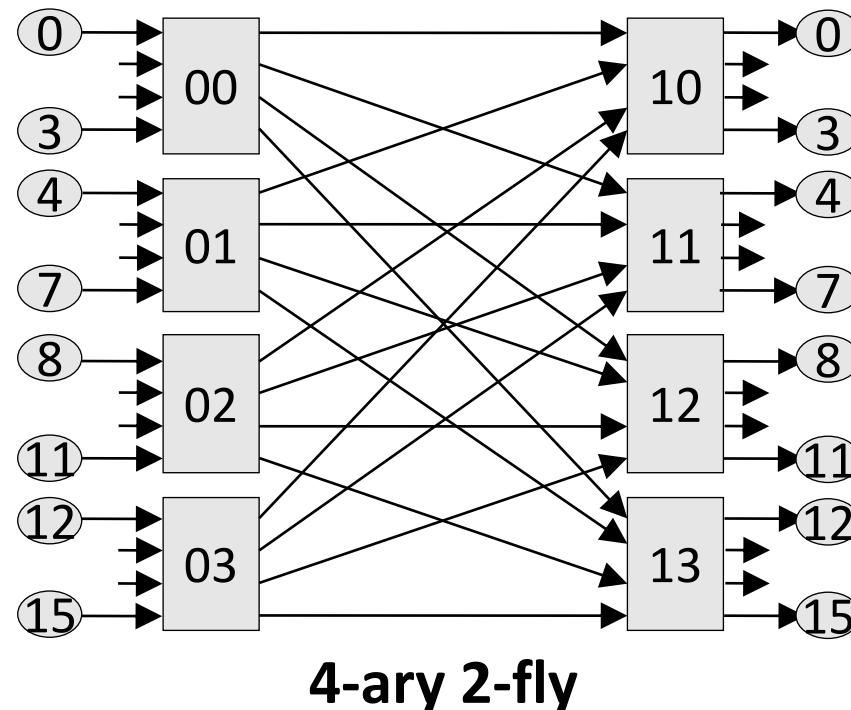
Destination-Tag Routing:

- Destination address directly routes packet in butterfly networks
- Destination address interpreted as n-digit radix-k number
- Each digit (starting from most-significant) selects the output port at each stage

e.g., Node 7 to Node 11:

Destination: $11_{10} = 23_4$

Therefore, use ports 2 then 3



Deterministic Routing – Destination-Tag

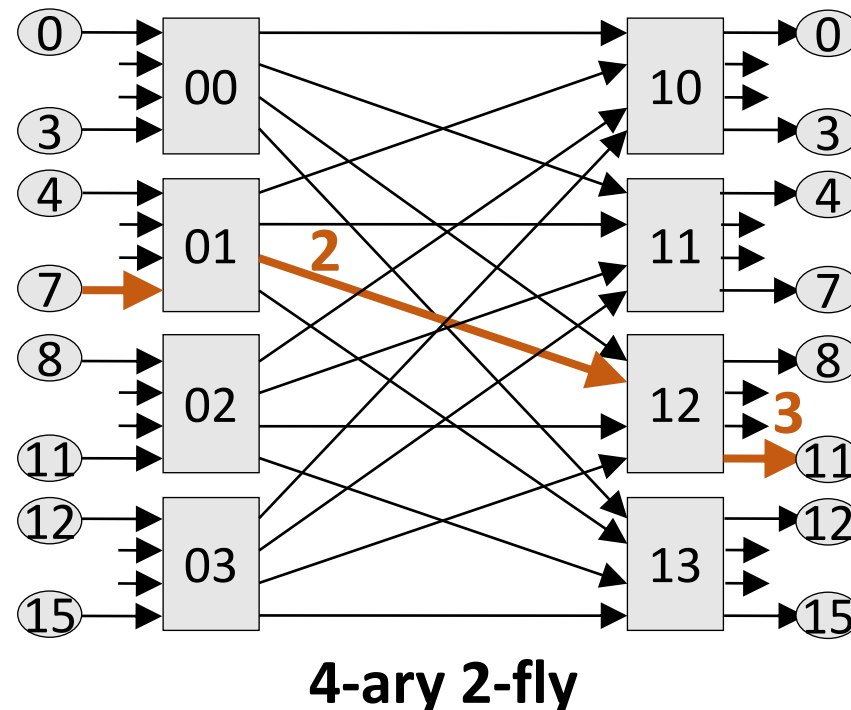
Destination-Tag Routing:

- Destination address directly routes packet in butterfly networks
- Destination address interpreted as n-digit radix-k number
- Each digit (starting from most-significant) selects the output port at each stage

e.g., Node 7 to Node 11:

Destination: $11_{10} = 23_4$

Therefore, use ports 2 then 3



Oblivious Routing

Routing decisions are made without regard to network state

- Keeps routing algorithms simple
- Unable to adapt to congestion on-the-fly

Deterministic routing algorithms are a subset of oblivious routing

Oblivious Routing – Valiant's Routing Algorithm

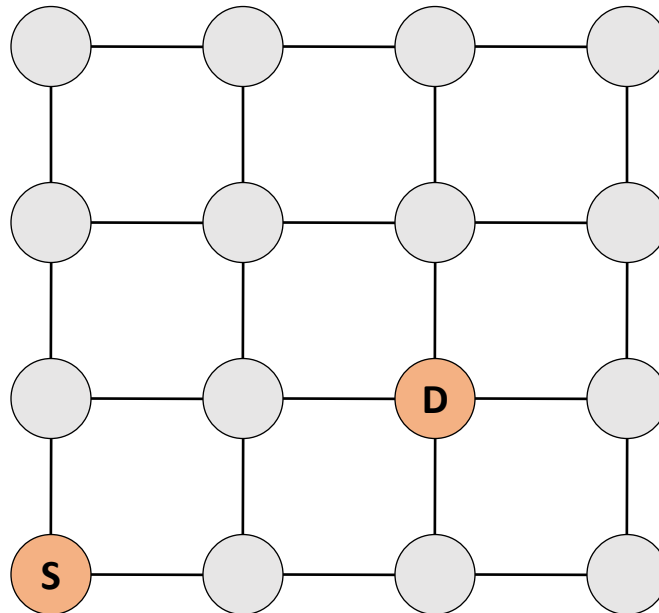
Randomizes any traffic pattern. To route from S to D:

- Randomly choose intermediate router D'
- Route from S to D' then from D' to D

Oblivious Routing – Valiant's Routing Algorithm

Randomizes any traffic pattern. To route from S to D:

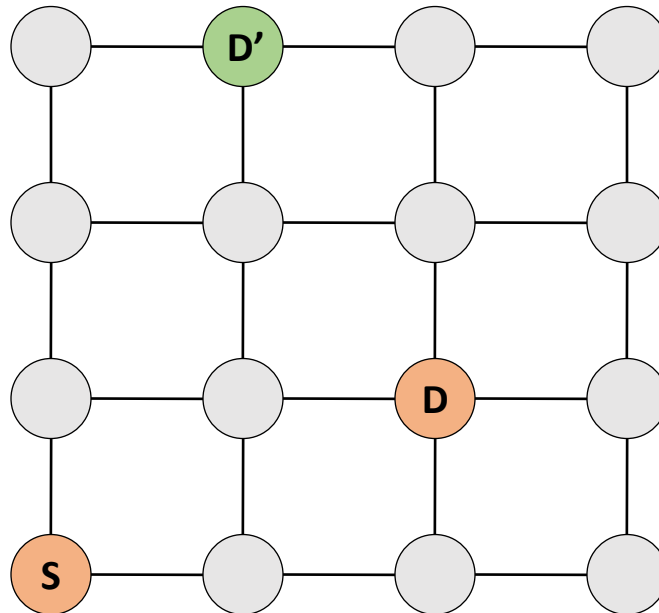
- Randomly choose intermediate router D'
- Route from S to D' then from D' to D



Oblivious Routing – Valiant's Routing Algorithm

Randomizes any traffic pattern. To route from S to D:

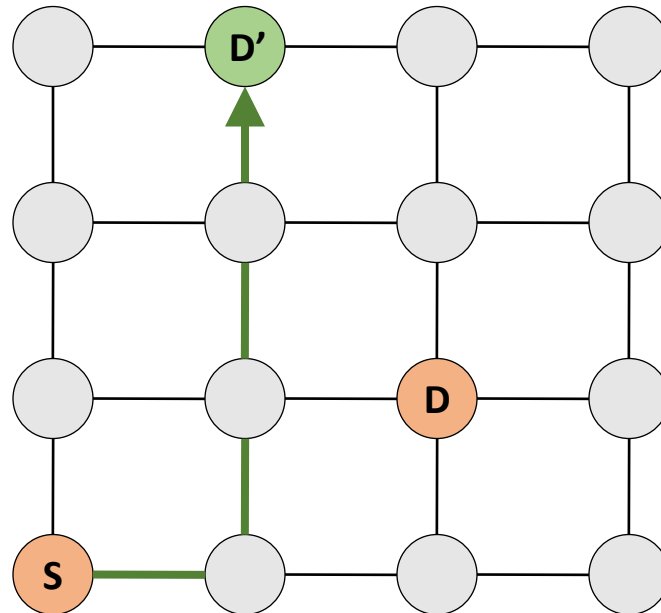
- Randomly choose intermediate router D'
- Route from S to D' then from D' to D



Oblivious Routing – Valiant's Routing Algorithm

Randomizes any traffic pattern. To route from S to D:

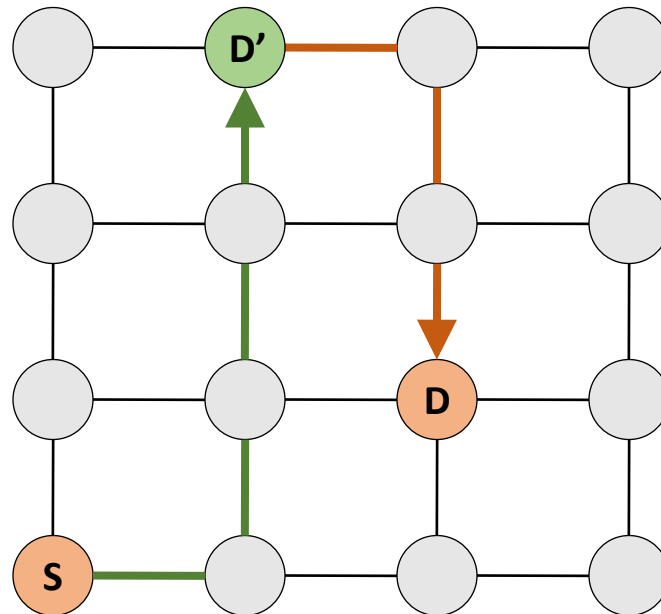
- Randomly choose intermediate router D'
- Route from S to D' then from D' to D



Oblivious Routing – Valiant's Routing Algorithm

Randomizes any traffic pattern. To route from S to D:

- Randomly choose intermediate router D'
- Route from S to D' then from D' to D



Oblivious Routing – Valiant's Routing Algorithm

Randomizes any traffic pattern. To route from S to D:

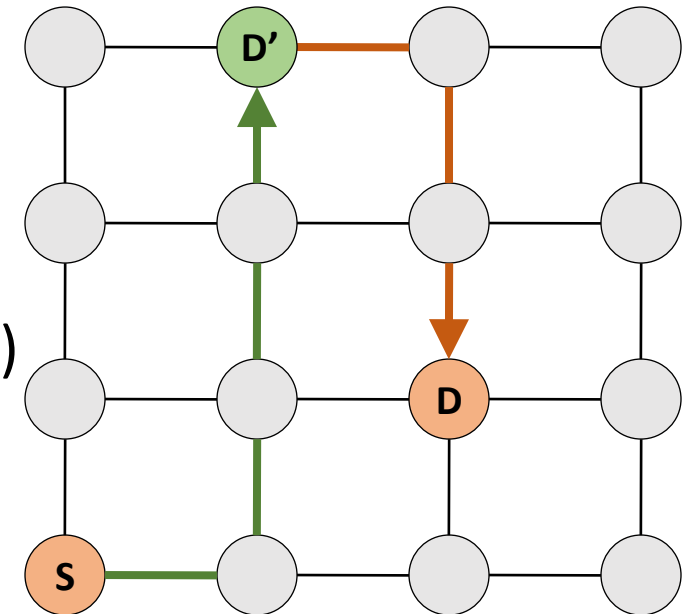
- Randomly choose intermediate router D'
- Route from S to D' then from D' to D

Disadvantages:

- Non-minimal routing (i.e., not shortest path)
- Destroys any locality in topology
- Introduces disallowed turns (needs deadlock-free flow control)

Advantages:

- Makes adversarial traffic patterns appear uniform random
- Balances network load



Oblivious Routing – Valiant's Routing Algorithm

For indirect networks:

- Two-pass Valiant's routing
- For butterfly network, equivalent to logically duplicating network

Oblivious Routing – Valiant's Routing Algorithm

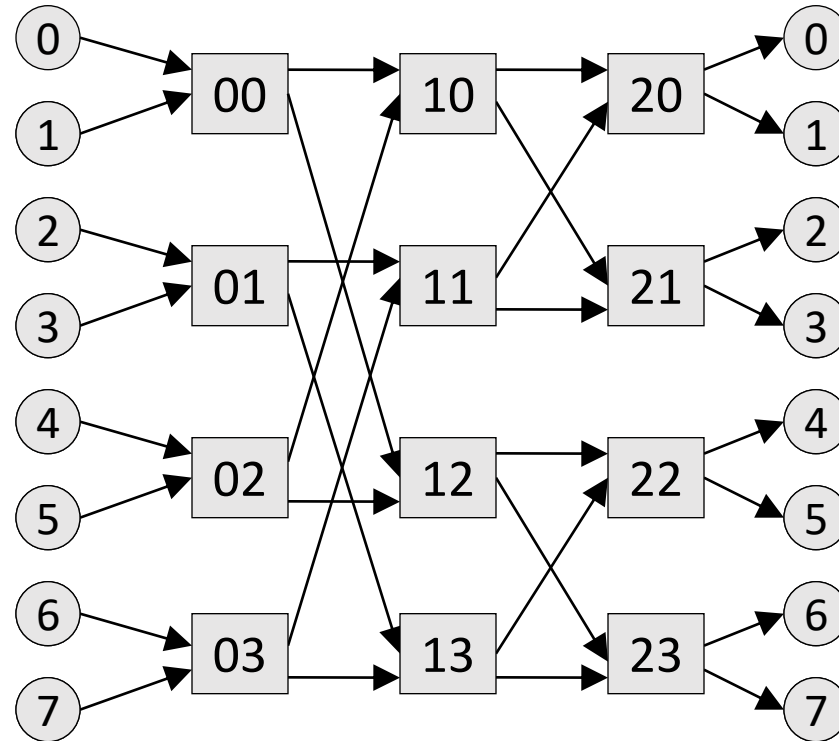
For indirect networks:

- Two-pass Valiant's routing
- For butterfly network, equivalent to logically duplicating network

e.g., Destination-Tag Routing:

Node 2 sends a lot to 7

Node 3 sends a lot to 5



Oblivious Routing – Valiant's Routing Algorithm

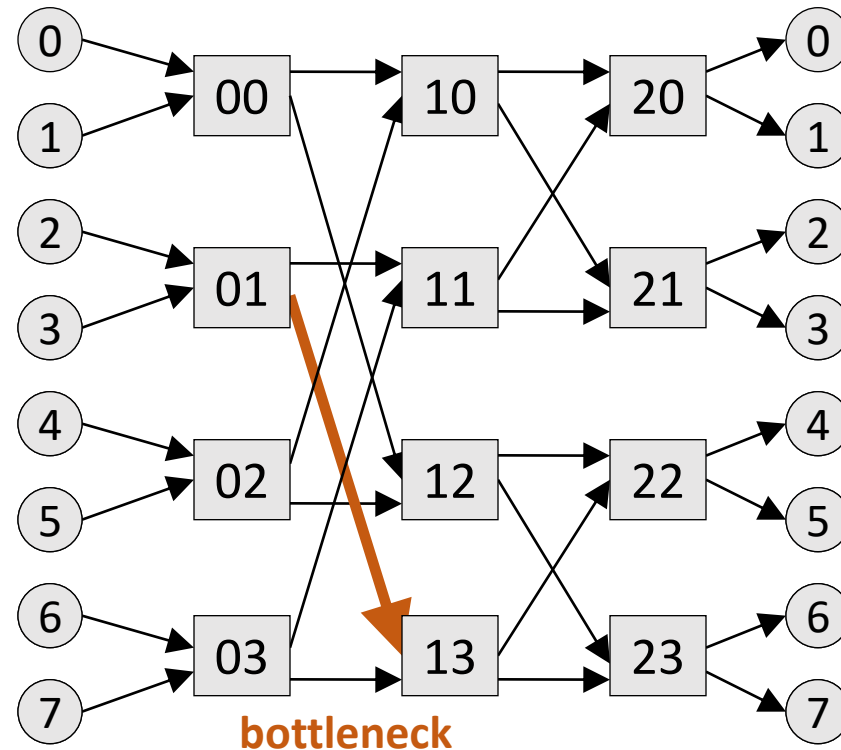
For indirect networks:

- Two-pass Valiant's routing
- For butterfly network, equivalent to logically duplicating network

e.g., Destination-Tag Routing:

Node 2 sends a lot to 7

Node 3 sends a lot to 5



Oblivious Routing – Valiant's Routing Algorithm

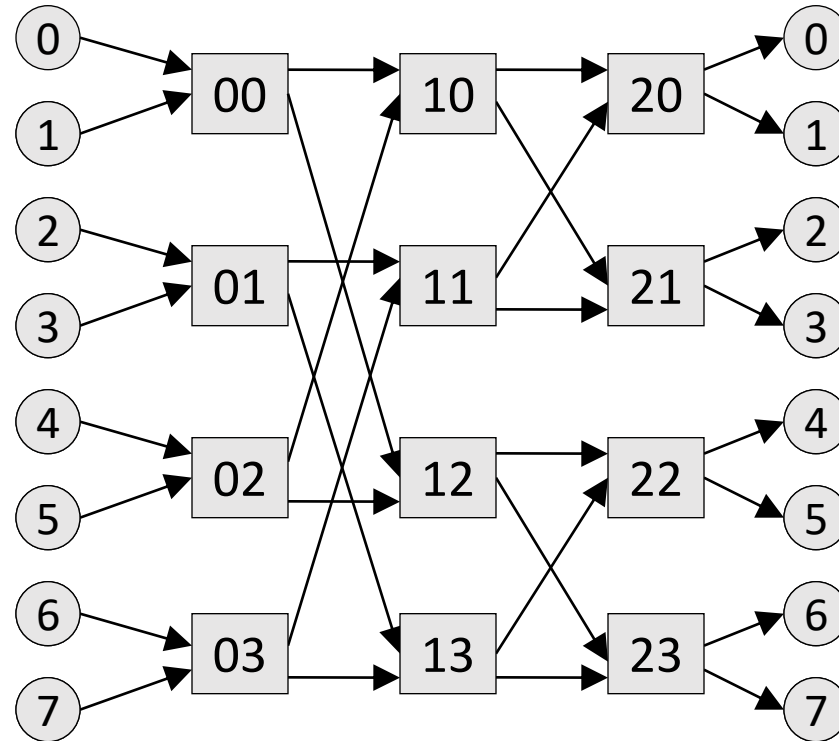
For indirect networks:

- Two-pass Valiant's routing
- For butterfly network, equivalent to logically duplicating network

e.g., Valiant's Routing:

Node 2 sends to 0 then to 7

Node 3 sends to 4 then to 5



Oblivious Routing – Valiant's Routing Algorithm

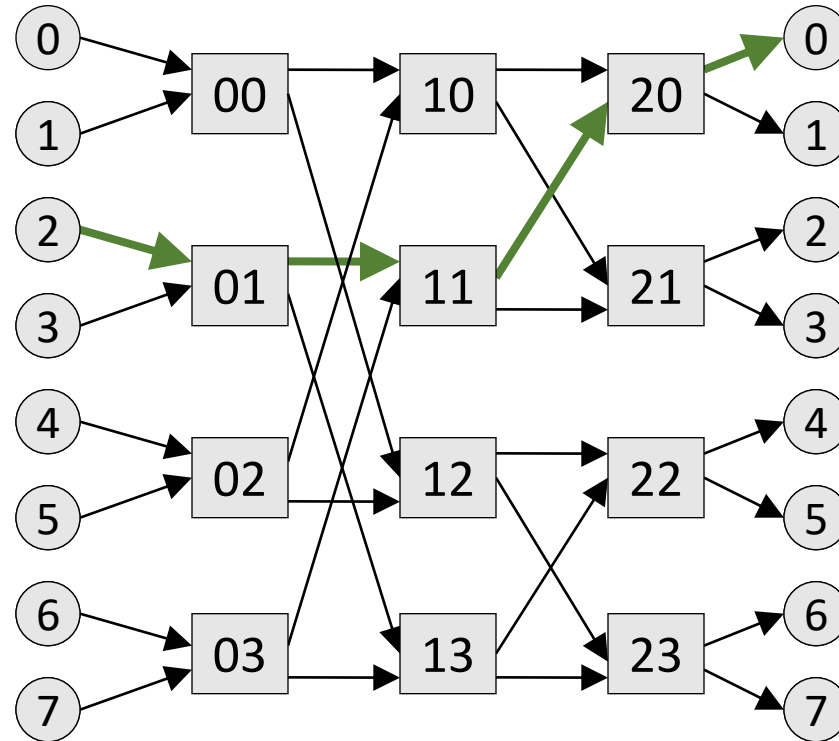
For indirect networks:

- Two-pass Valiant's routing
- For butterfly network, equivalent to logically duplicating network

e.g., Valiant's Routing:

Node 2 sends to 0 then to 7

Node 3 sends to 4 then to 5



Oblivious Routing – Valiant's Routing Algorithm

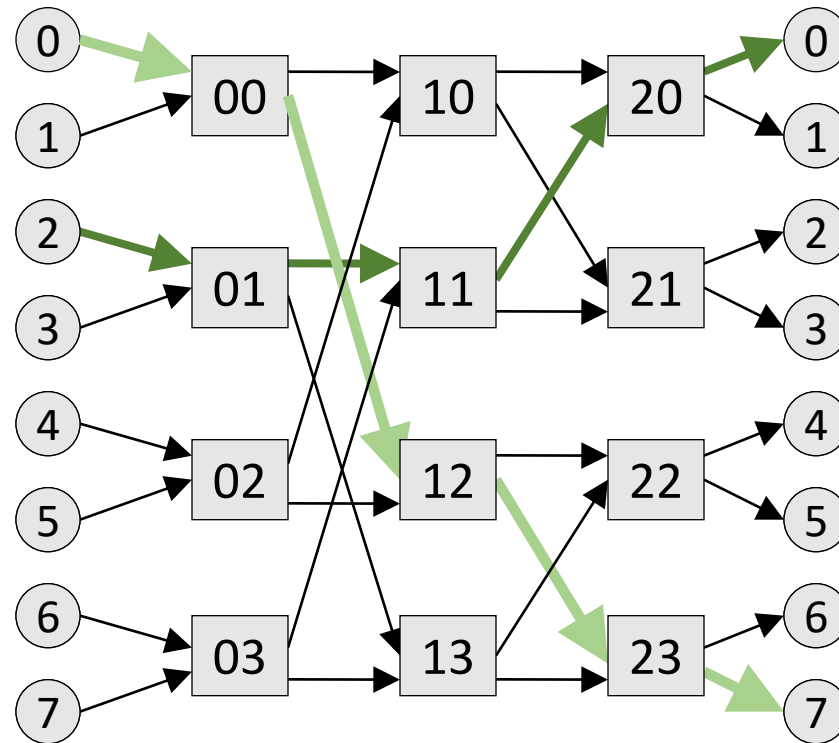
For indirect networks:

- Two-pass Valiant's routing
- For butterfly network, equivalent to logically duplicating network

e.g., Valiant's Routing:

Node 2 sends to 0 then to 7

Node 3 sends to 4 then to 5



Oblivious Routing – Valiant's Routing Algorithm

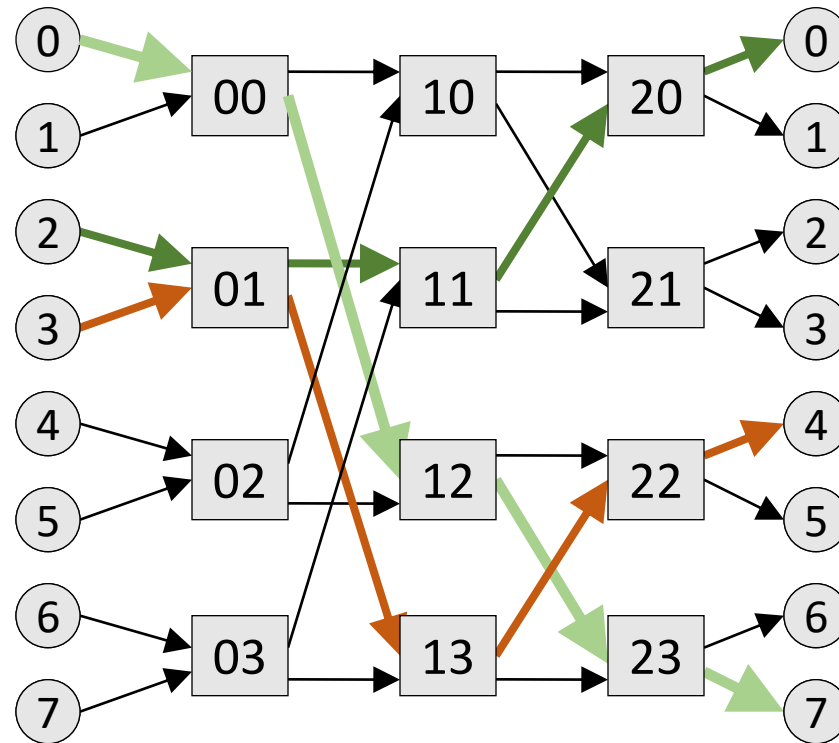
For indirect networks:

- Two-pass Valiant's routing
- For butterfly network, equivalent to logically duplicating network

e.g., Valiant's Routing:

Node 2 sends to 0 then to 7

Node 3 sends to 4 then to 5



Oblivious Routing – Valiant's Routing Algorithm

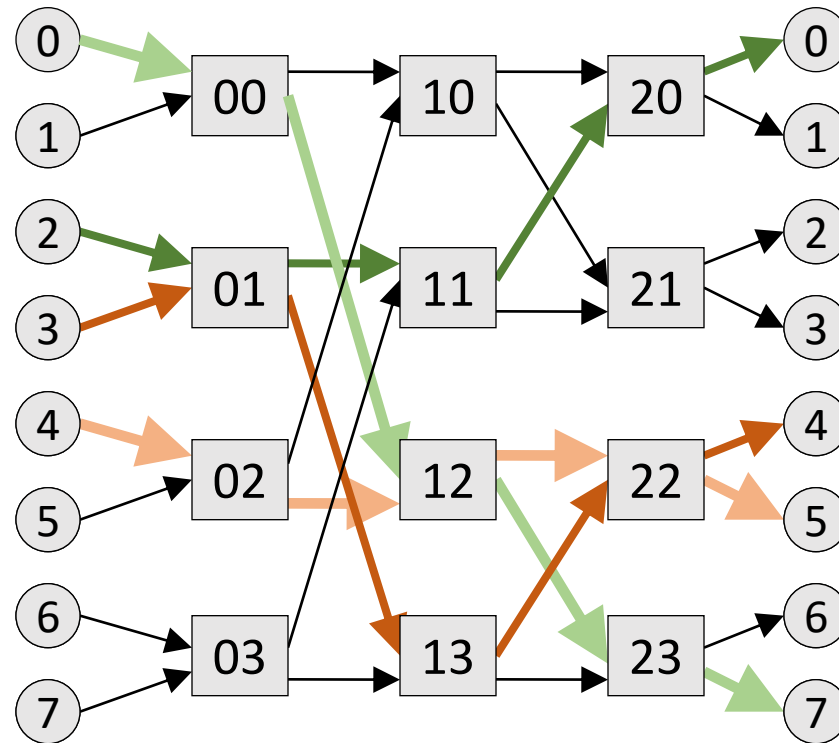
For indirect networks:

- Two-pass Valiant's routing
- For butterfly network, equivalent to logically duplicating network

e.g., Valiant's Routing:

Node 2 sends to 0 then to 7

Node 3 sends to 4 then to 5



Oblivious Routing – Valiant's Routing Algorithm

For indirect networks:

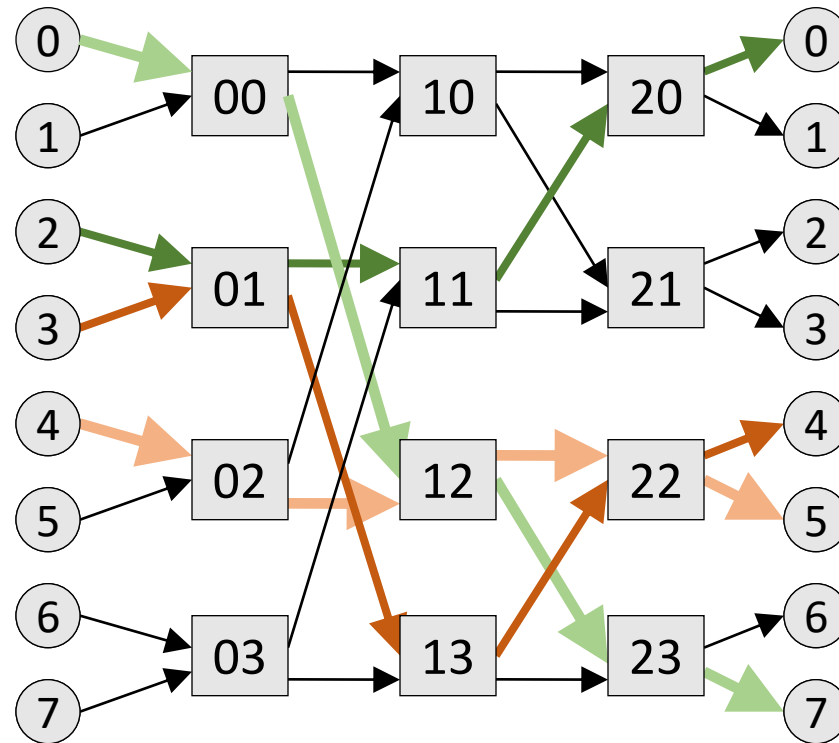
- Two-pass Valiant's routing
- For butterfly network, equivalent to logically duplicating network

e.g., Valiant's Routing:

Node 2 sends to 0 then to 7

Node 3 sends to 4 then to 5

Distributes traffic



Oblivious Routing – Minimal Oblivious

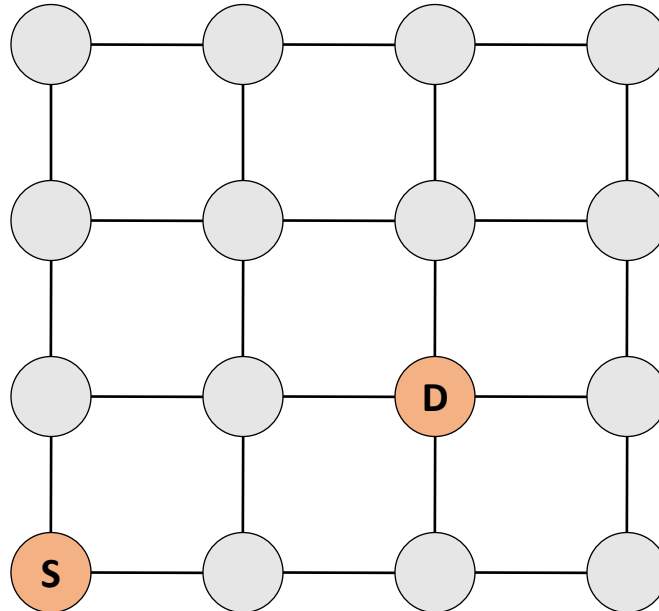
Minimal Oblivious Routing:

- Apply Valiant's routing to minimal quadrant between S and D
- Shortest path routing, but less load balancing than Valiant's

Oblivious Routing – Minimal Oblivious

Minimal Oblivious Routing:

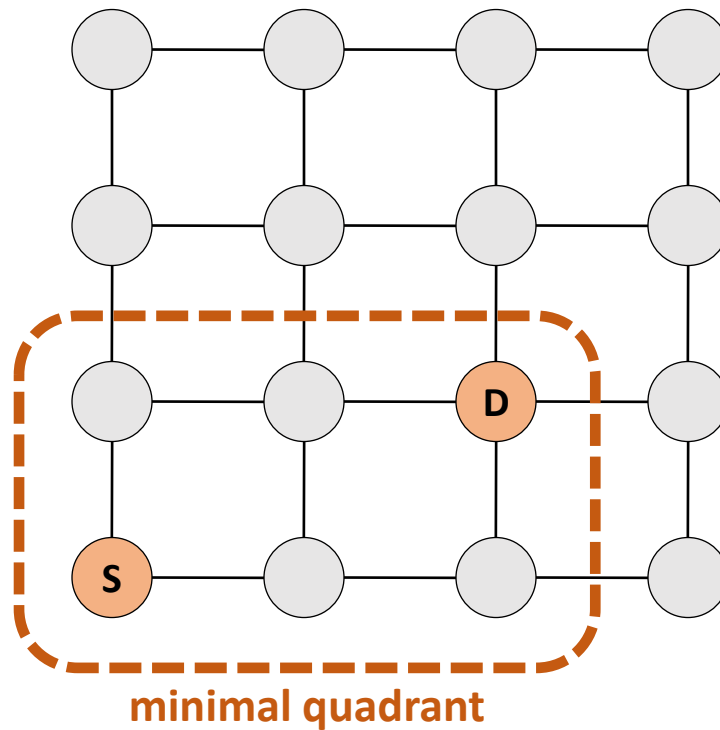
- Apply Valiant's routing to minimal quadrant between S and D
- Shortest path routing, but less load balancing than Valiant's



Oblivious Routing – Minimal Oblivious

Minimal Oblivious Routing:

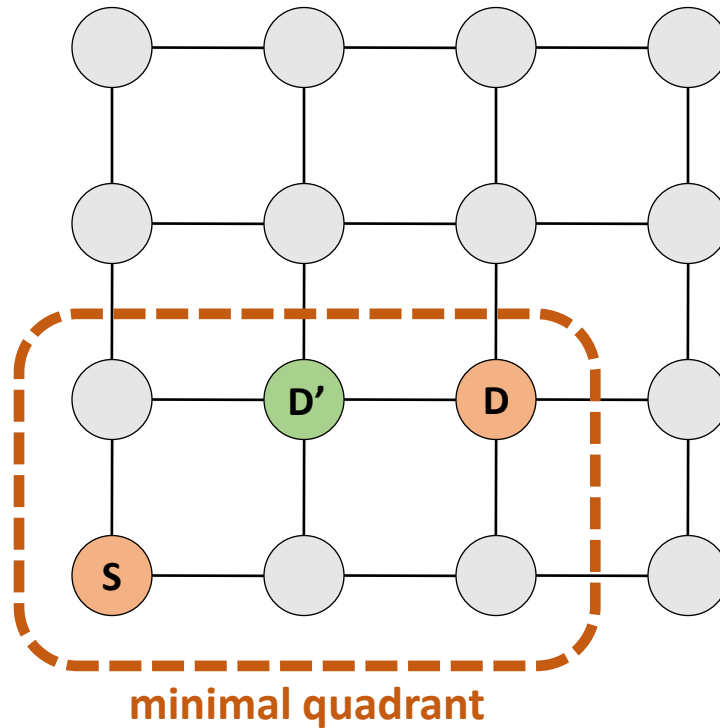
- Apply Valiant's routing to minimal quadrant between S and D
- Shortest path routing, but less load balancing than Valiant's



Oblivious Routing – Minimal Oblivious

Minimal Oblivious Routing:

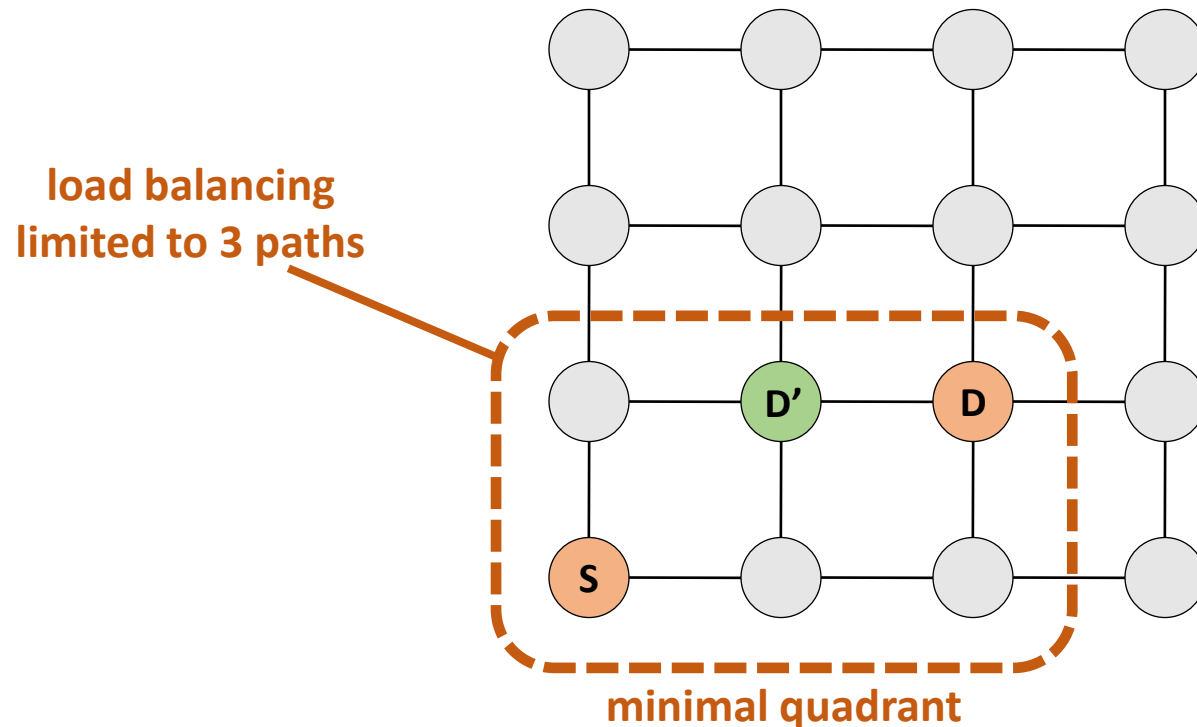
- Apply Valiant's routing to minimal quadrant between S and D
- Shortest path routing, but less load balancing than Valiant's



Oblivious Routing – Minimal Oblivious

Minimal Oblivious Routing:

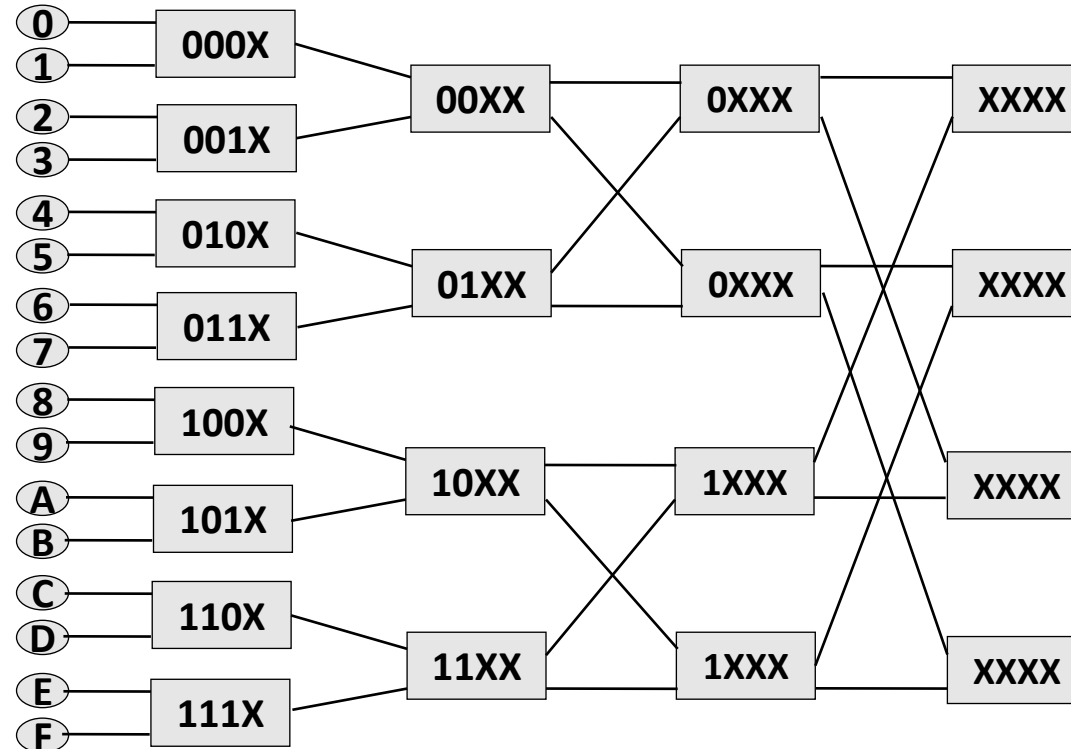
- Apply Valiant's routing to minimal quadrant between S and D
- Shortest path routing, but less load balancing than Valiant's



Oblivious Routing – Minimal Oblivious

For fat tree networks, to route from S to D:

- For each upward hop, randomly select output port to parent router
- If current router (partially) matches destination address, go downward



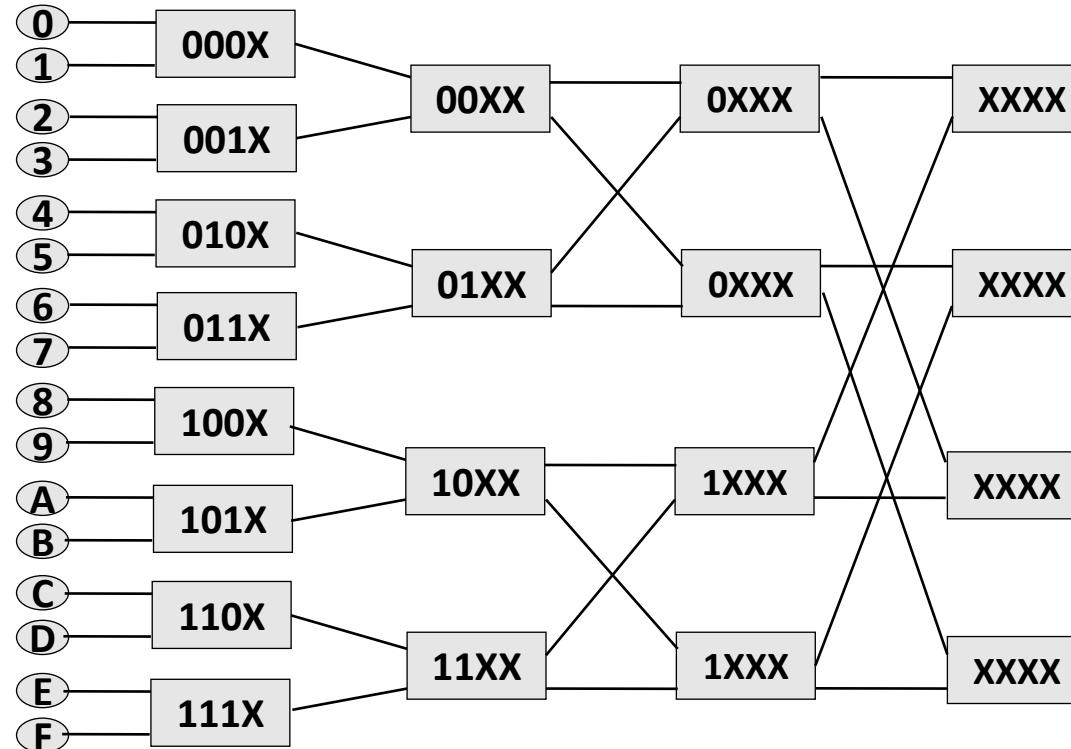
Oblivious Routing – Minimal Oblivious

For fat tree networks, to route from S to D:

- For each upward hop, randomly select output port to parent router
- If current router (partially) matches destination address, go downward

e.g., Node 1 to 6:

Upward until match 0110_2



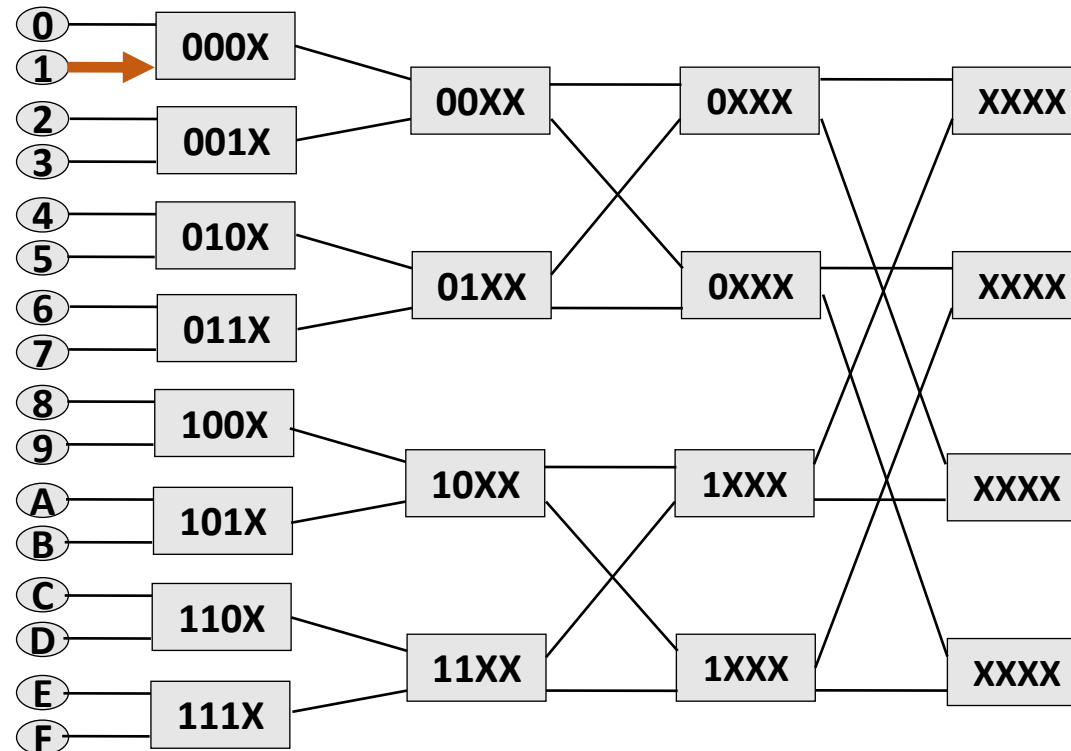
Oblivious Routing – Minimal Oblivious

For fat tree networks, to route from S to D:

- For each upward hop, randomly select output port to parent router
- If current router (partially) matches destination address, go downward

e.g., Node 1 to 6:

Upward until match 0110_2

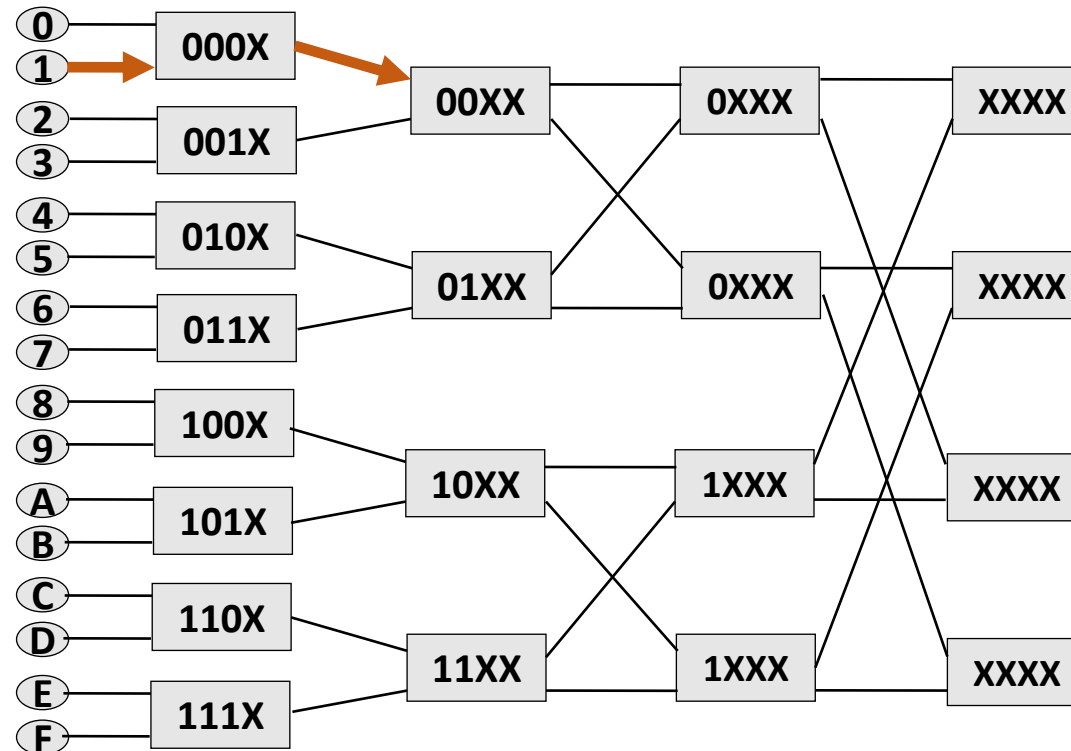


Oblivious Routing – Minimal Oblivious

For fat tree networks, to route from S to D:

- For each upward hop, randomly select output port to parent router
- If current router (partially) matches destination address, go downward

e.g., Node 1 to 6:
Upward until match 0110_2



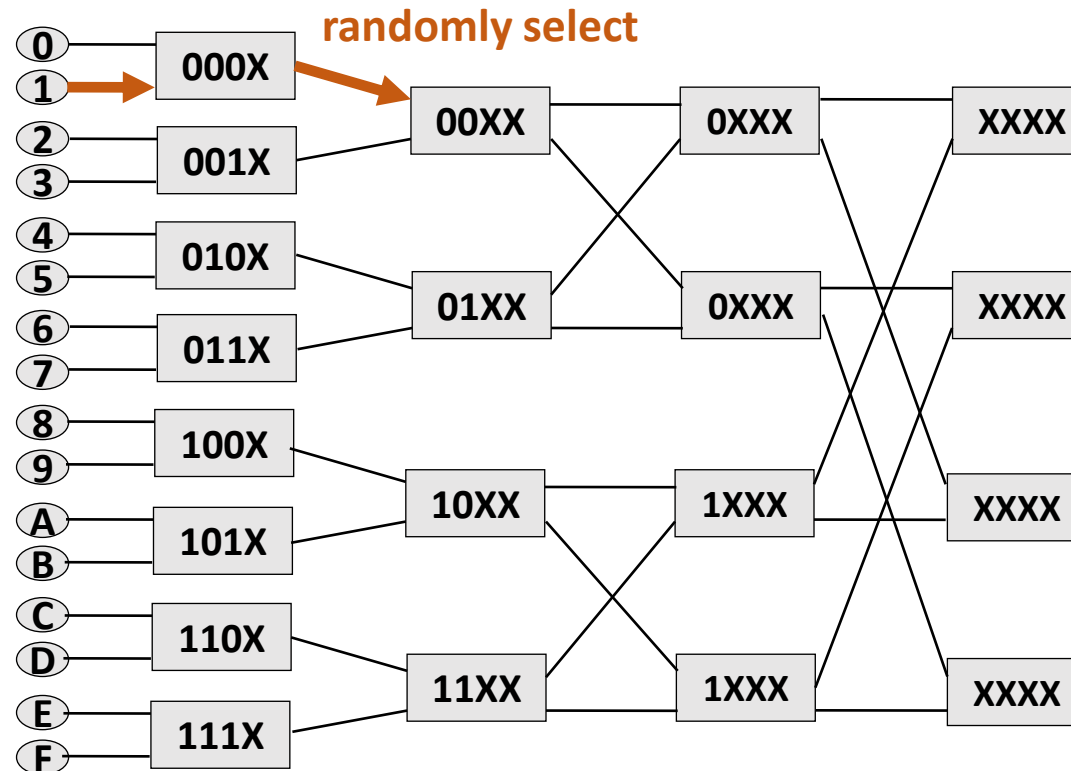
Oblivious Routing – Minimal Oblivious

For fat tree networks, to route from S to D:

- For each upward hop, randomly select output port to parent router
- If current router (partially) matches destination address, go downward

e.g., Node 1 to 6:

Upward until match 0110_2

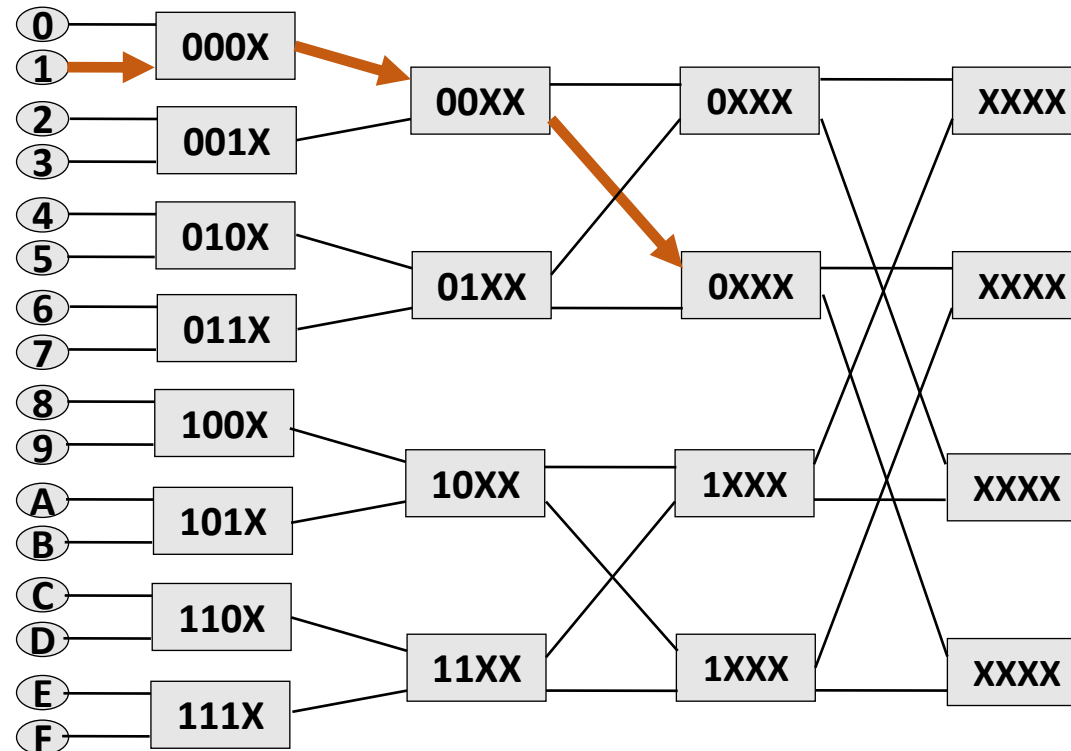


Oblivious Routing – Minimal Oblivious

For fat tree networks, to route from S to D:

- For each upward hop, randomly select output port to parent router
- If current router (partially) matches destination address, go downward

e.g., Node 1 to 6:
Upward until match 0110_2



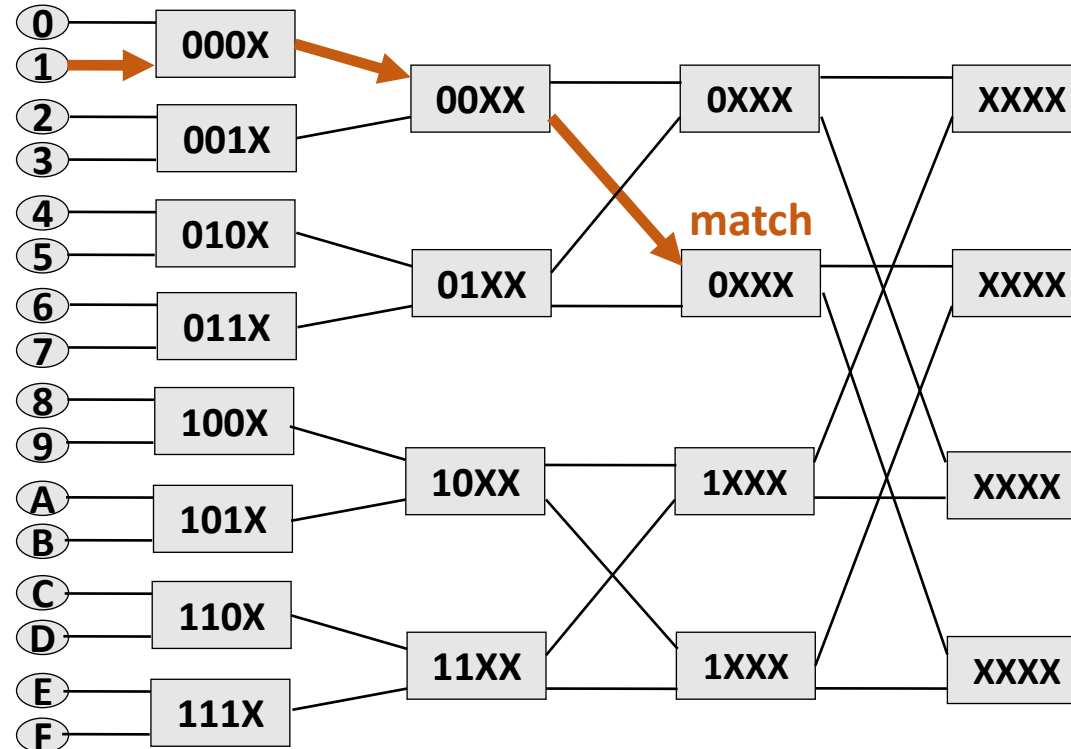
Oblivious Routing – Minimal Oblivious

For fat tree networks, to route from S to D:

- For each upward hop, randomly select output port to parent router
- If current router (partially) matches destination address, go downward

e.g., Node 1 to 6:

Upward until match 0110_2



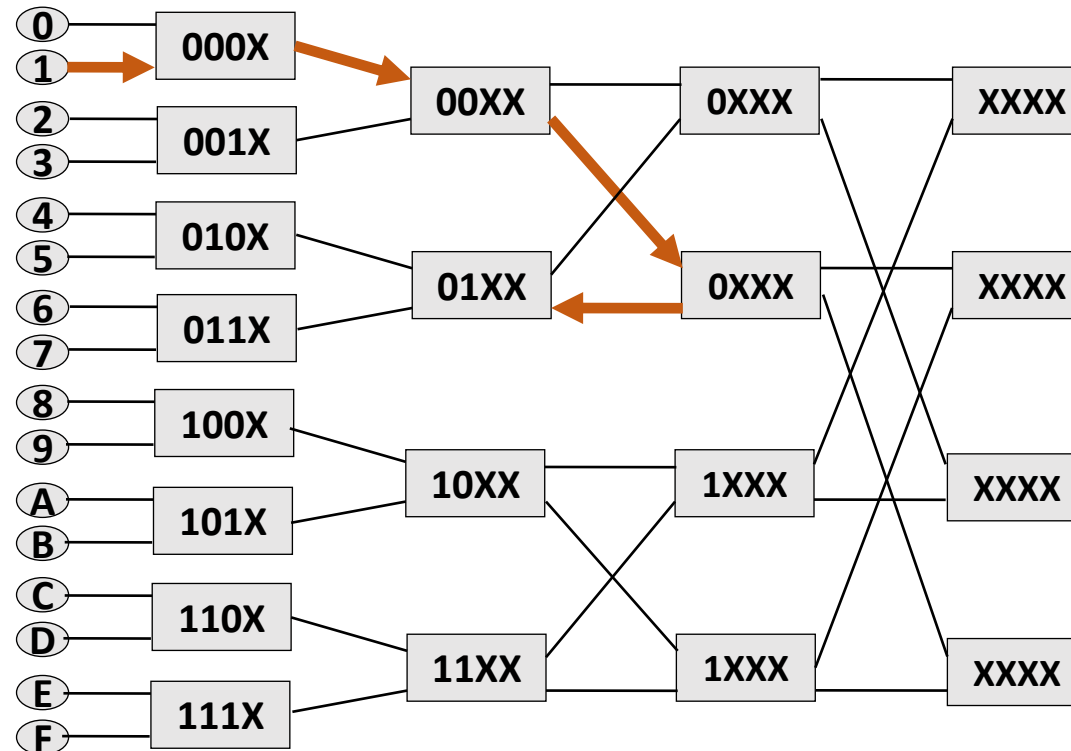
Oblivious Routing – Minimal Oblivious

For fat tree networks, to route from S to D:

- For each upward hop, randomly select output port to parent router
- If current router (partially) matches destination address, go downward

e.g., Node 1 to 6:

Upward until match 0110_2

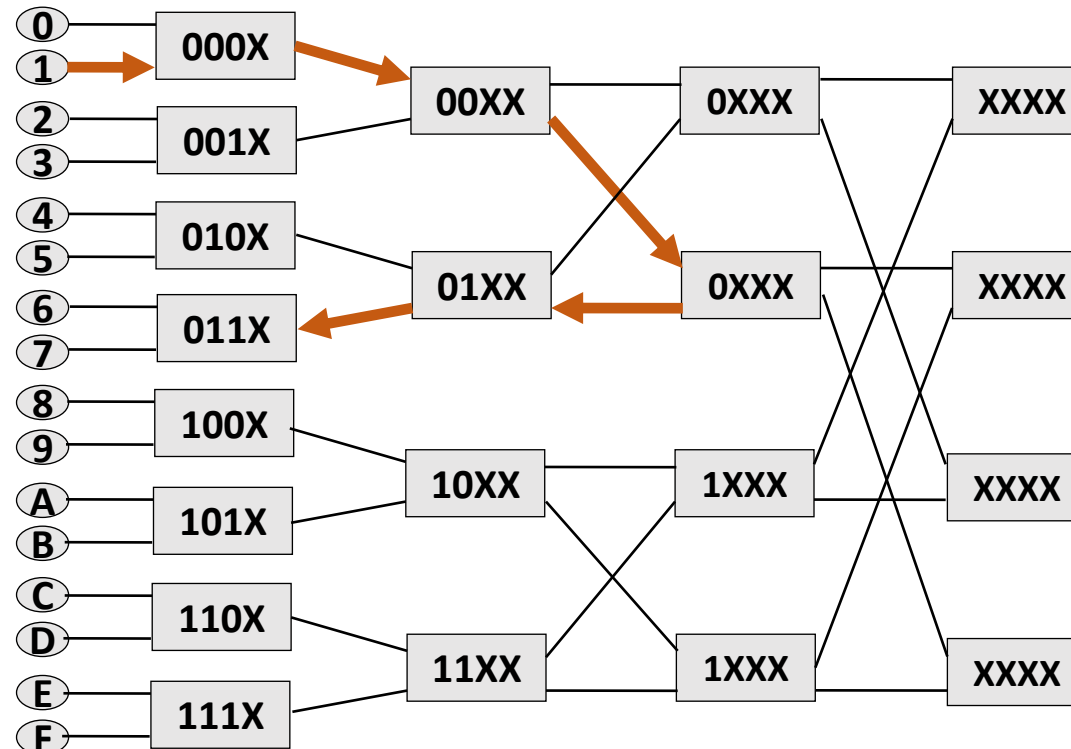


Oblivious Routing – Minimal Oblivious

For fat tree networks, to route from S to D:

- For each upward hop, randomly select output port to parent router
- If current router (partially) matches destination address, go downward

e.g., Node 1 to 6:
Upward until match 0110_2

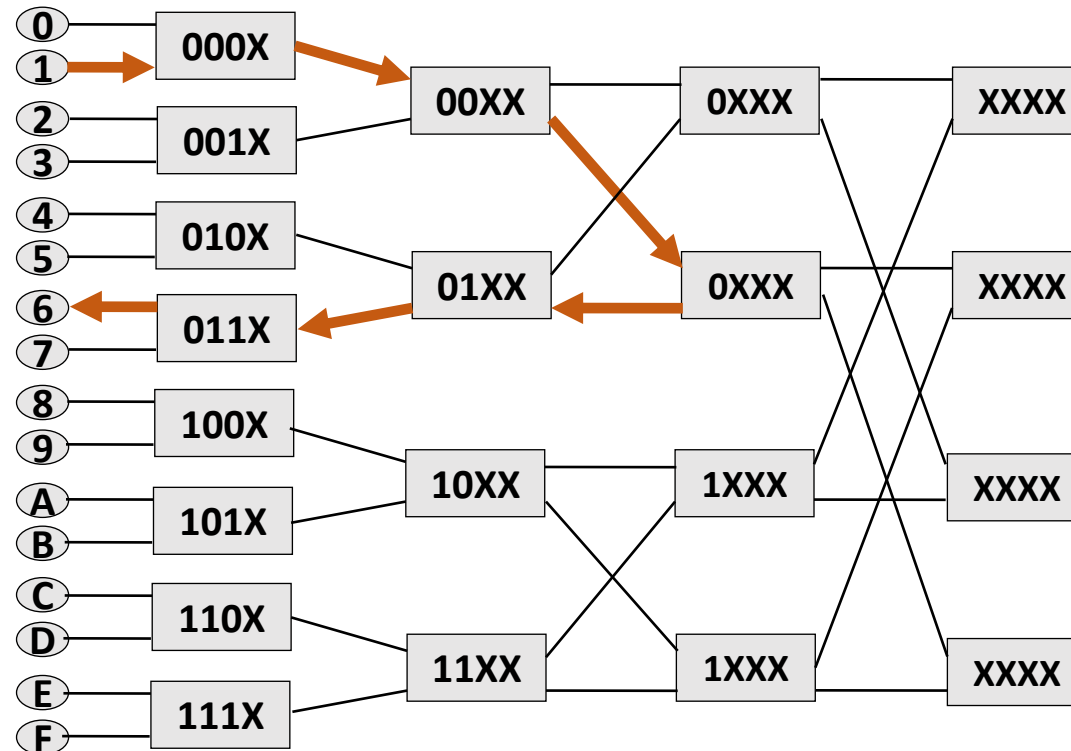


Oblivious Routing – Minimal Oblivious

For fat tree networks, to route from S to D:

- For each upward hop, randomly select output port to parent router
- If current router (partially) matches destination address, go downward

e.g., Node 1 to 6:
Upward until match 0110_2



Oblivious Routing – Minimal Oblivious

For fat tree networks, to route from S to D:

- For each upward hop, randomly select output port to parent router
- If current router (partially) matches destination address, go downward

e.g., Node 1 to 6:
Upward until match 0110_2
Path diversity = 2

